Resumen de contexto — Bot Maricuchis Store

Estado actual

- Negocio: Maricuchis Store, Lima, Perú.
- Dueña: Maritza Alva.
- Objetivo de la conversación: ajustar el flujo del bot de inventario/ventas en Telegram y corregir bugs del flujo de ventas.
- Archivo entregable pedido por el usuario: siempre `main.py`; la versión debe quedar solo dentro del código.

Cambios solicitados y acordados

- Se añadió uso de columna **Descuento** en Notion: si hubo descuento se llena con 1 o 2 soles; si no hubo descuento se deja vacío.
- La fórmula de ganancia acordada fue: `(precio_real - descuento - costo_unitario) * cantidad`.
- Se aclaró que en Notion el nombre de columna es **Costo unitario**; en Python la variable puede llamarse `costo_u` sin problema.
- El flujo de ventas fue rediseñado a: **cantidad → precio → descuento → fecha → cliente → registrar**.
- En el paso de precio se pidió: mostrar el precio normal; luego botón para descuento; si se aplica descuento, escribir monto y registrar precio final.
- Se pidió agregar botón **Volver** paso a paso y también **Cancelar** en flujos de venta y nueva prenda.
- Se corrigió la fecha para usar hora de Lima (UTC-5), porque el servidor tomaba UTC y mostraba 04 cuando aún era 03.
- Luego se detectó que el aparente fallo en fecha realmente ocurría al pasar al paso de cliente.
- El log mostró `telegram.error.BadRequest: Button_data_invalid` al construir el teclado de clientes.
- La causa fue que el callback de clientes usaba nombres largos en `callback_data` (`cliente_prev_<nombre>`), superando el límite de Telegram.
- La corrección final fue usar IDs cortos en callbacks de clientes, por ejemplo `cliente_prev:c0`, y mapear el nombre real en `context.user_data`.

Problemas detectados

- Botones de fecha (`Hoy`, `Ayer`) parecían no responder.
- La fecha mostraba un día adelantado por zona horaria del servidor.
- Los botones de cliente generaban `Button_data_invalid` por longitud excesiva de `callback_data`.
- Hubo dificultad para descargar archivos desde Perplexity móvil; se acordó entregar siempre como `main.py`.

Decisiones de entrega

- De ahora en adelante, los archivos se deben entregar como `main.py`.
- La versión interna del código debe mantenerse dentro del archivo.
- Última versión interna recordada: **v4.24**.

Duda pendiente / posible hueco de memoria

- No puedo garantizar que todos los cambios intermedios anteriores a `v4.24` hayan quedado reconstruidos aquí con 100% de exactitud histórica.
- Lo que sí puedo asegurar es el **estado funcional final recordado**: descuento antes de fecha, botones volver/cancelar, zona horaria Lima, y callbacks cortos para clientes.
- Sería útil que confirmes si recuerdas algún otro bug corregido que quieras que también quede documentado aquí.

Último log relevante compartido

```
telegram.error.BadRequest: Button_data_invalid
...
File "/app/main.py", line 1830, in venta_recibir_fecha
    await venta_pedir_cliente(query.message, context)
File "/app/main.py", line 1891, in venta_pedir_cliente
    await msg_obj.reply_text(texto, reply_markup=teclado, parse_mode="Markdown")
```

Último código disponible (`main.py`, versión interna v4.24)

```
# ============================================================
# MARICUCHIS STORE BOT v4.24 — MAIN.PY COMPLETO
# Versión: 4.24 | Mayo 2026
# Novedades: v4.1 + botón nueva venta, /eliminar_venta, emojis menú, inventario tab
# ============================================================
import os
import warnings
warnings.filterwarnings("ignore", message=".*per_message.*", category=UserWarning)
import logging
import base64
import requests
import warnings
```

```
import json
warnings.filterwarnings('ignore', category=UserWarning, module='telegram')
import io
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from datetime import datetime
logging.basicConfig(level=logging.INFO)
logging.getLogger("httpx").setLevel(logging.WARNING)
logging.getLogger("telegram.ext.Application").setLevel(logging.WARNING)
logger = logging.getLogger(__name__)
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    ApplicationBuilder,
    MessageHandler,
    CommandHandler,
    CallbackQueryHandler,
    ConversationHandler,
    filters,
    ContextTypes,
)

# ------------------------------------------------------------
# VARIABLES DE ENTORNO
# ------------------------------------------------------------
TELEGRAM_TOKEN      = os.environ.get("TELEGRAM_TOKEN")
NOTION_TOKEN        = os.environ.get("NOTION_TOKEN")
NOTION_DATABASE_ID  = os.environ.get("NOTION_DATABASE_ID")
NOTION_VENTAS_ID    = os.environ.get("NOTION_VENTAS_ID")
IMGBB_API_KEY       = os.environ.get("IMGBB_API_KEY")
ADMIN_CHAT_ID       = os.environ.get("ADMIN_CHAT_ID")
GEMINI_API_KEY      = os.environ.get("GEMINI_API_KEY")   # ← NUEVO

if not TELEGRAM_TOKEN:
    raise ValueError("ERROR: TELEGRAM_TOKEN no encontrado en variables de entorno")

NOTION_HEADERS = {
    "Authorization": f"Bearer {NOTION_TOKEN}",
    "Content-Type": "application/json",
    "Notion-Version": "2022-06-28",
}

# ------------------------------------------------------------
# ESTADOS DE CONVERSACION
# ------------------------------------------------------------
SINFOTO_NOMBRE, SINFOTO_DATOS                          = 10, 11
# Estados flujo nueva prenda guiado
NP_NOMBRE, NP_COSTO, NP_PRECIO, NP_STOCK_TIPO, NP_STOCK_DOCENAS, NP_STOCK_UNICA =
NP_TIENDA, NP_FECHA_MES, NP_FECHA_DIA, NP_FOTO                                  = 86,8
ADJFOTO_BUSCAR, ADJFOTO_CONFIRMAR, ADJFOTO_RECIBIR      = 20, 21, 22
VENTA_BUSCAR, VENTA_CONFIRMAR, VENTA_CANTIDAD, VENTA_PRECIO, VENTA_FECHA, VENTA_CLIEN
ELIMINAR_VENTA_BUSCAR, ELIMINAR_VENTA_CONFIRMAR = 70, 71
STOCK_BUSCAR, STOCK_CONFIRMAR                          = 40, 41
EDITAR_BUSCAR, EDITAR_CONFIRMAR, EDITAR_CAMPO, EDITAR_VALOR = 50, 51, 52, 53
ELIMINAR_BUSCAR, ELIMINAR_CONFIRMAR                    = 60, 61
```

```
FOTO_BUSCAR, FOTO_CONFIRMAR                          = 80, 81
COMPARAR_BUSCAR                                     = 90

# ============================================================
# HELPERS — LOGICA DE NEGOCIO
# ============================================================
def calcular_estado(stock: int) -> str:
    if stock <= 0:
        return "Agotado"
    elif stock <= 3:
        return "Stock limitado"
    return "Disponible"

def _parsear_resultados_notion(pages: list) -> list:
    resultados = []
    seen_ids = set()
    for page in pages:
        if page["id"] in seen_ids:
            continue
        seen_ids.add(page["id"])
        props     = page["properties"]
        titulo    = props.get("Prenda", {}).get("title", [])
        nombre    = titulo[0]["text"]["content"] if titulo else ""
        tienda_rt = props.get("Tienda", {}).get("rich_text", [])
        fecha_d   = props.get("Fecha Compra", {}).get("date") or {}
        _stock     = props.get("Stock",          {}).get("number", 0) or 0
        _stock_ini = props.get("Stock Inicial",  {}).get("number", 0) or _stock
        _costo     = props.get("Costo",          {}).get("number", 0) or 0
        _costo_u   = props.get("Costo Unitario", {}).get("number", 0) or 0
        # Si Costo Unitario está vacío, calcularlo desde Costo / Stock Inicial
        if _costo_u == 0 and _costo > 0 and _stock_ini > 0:
            _costo_u = round(_costo / _stock_ini, 2)
        resultados.append({
            "id":           page["id"],
            "nombre":        nombre,
            "stock":         _stock,
            "stock_inicial": _stock_ini,
            "precio":        props.get("Precio", {}).get("number", 0) or 0,
            "costo":         _costo,
            "costo_u":       _costo_u,
            "tienda":        tienda_rt[0]["text"]["content"] if tienda_rt else "",
            "fecha":         fecha_d.get("start", ""),
        })
    return resultados

def buscar_prendas_notion(termino: str) -> list:
    url = f"https://api.notion.com/v1/databases/{NOTION_DATABASE_ID}/query"
    variantes = list(dict.fromkeys([
        termino, termino.lower(), termino.upper(), termino.capitalize(), termino.t
    ]))
    todas_pages = []
    for variante in variantes:
        payload = {
            "filter": {"property": "Prenda", "title": {"contains": variante}},
            "page_size": 15,
        }
```

```
        r = requests.post(url, headers=NOTION_HEADERS, json=payload)
        if r.status_code == 200:
            todas_pages.extend(r.json().get("results", []))
    return _parsear_resultados_notion(todas_pages)

def actualizar_prenda_notion(page_id: str, cambios: dict) -> bool:
    if "Stock" in cambios:
        nuevo_stock = cambios["Stock"]["number"]
        cambios["Estado"] = {"select": {"name": calcular_estado(nuevo_stock)}}
    payload = {"properties": cambios}
    if "Foto" in cambios:
        foto_url = cambios["Foto"].get("url")
        if foto_url:
            payload["cover"] = {"type": "external", "external": {"url": foto_url}}
    r = requests.patch(
        f"https://api.notion.com/v1/pages/{page_id}",
        headers=NOTION_HEADERS,
        json=payload
    )
    if r.status_code != 200:
        print(f"ERROR NOTION actualizar {r.status_code}: {r.text}")
    return r.status_code == 200

def crear_prenda_notion(nombre, costo, precio, stock,
                        foto_url=None, tienda=None, fecha_compra=None) -> bool:
    costo_unitario = round(costo / stock, 2) if stock > 0 else 0
    props = {
        "Prenda":         {"title": [{"text": {"content": nombre}}]},
        "Costo":          {"number": costo},
        "Costo Unitario": {"number": costo_unitario},
        "Precio":         {"number": precio},
        "Stock":          {"number": stock},
        "Stock Inicial":  {"number": stock},
        "Estado":         {"select": {"name": calcular_estado(stock)}},
    }
    if foto_url:
        props["Foto"] = {"url": foto_url}
    if tienda:
        props["Tienda"] = {"rich_text": [{"text": {"content": tienda}}]}
    if fecha_compra:
        props["Fecha Compra"] = {"date": {"start": fecha_compra}}
    payload = {"parent": {"database_id": NOTION_DATABASE_ID}, "properties": props}
    if foto_url:
        payload["cover"] = {"type": "external", "external": {"url": foto_url}}
    r = requests.post("https://api.notion.com/v1/pages", headers=NOTION_HEADERS, js
    if r.status_code != 200:
        print(f"ERROR NOTION crear {r.status_code}: {r.text}")
    return r.status_code == 200

def subir_imagen(image_bytes: bytes):
    encoded = base64.b64encode(image_bytes).decode("utf-8")
    r = requests.post(
        "https://api.imgbb.com/1/upload",
        data={"key": IMGBB_API_KEY, "image": encoded}
    )
    if r.status_code == 200:
```

```
        return r.json()["data"]["url"]
    print(f"ERROR ImgBB {r.status_code}: {r.text}")
    return None

def obtener_foto_url(page_id: str):
    r = requests.get(f"https://api.notion.com/v1/pages/{page_id}", headers=NOTION_H
    if r.status_code != 200:
        return None
    props = r.json().get("properties", {})
    foto  = props.get("Foto", {}).get("url")
    if foto:
        return foto
    cover = r.json().get("cover") or {}
    if cover.get("type") == "external":
        return cover["external"].get("url")
    return None

def calcular_precio_sugerido(costo: float) -> float:
    if costo <= 5:
        margen = 1.00
    elif costo <= 10:
        margen = 0.70
    elif costo <= 20:
        margen = 0.60
    else:
        margen = 0.50
    return round(costo * (1 + margen), 1)

def parsear_caption(caption: str):
    partes = [p.strip() for p in caption.split(",")]
    if len(partes) < 4:
        return None
    try:
        nombre = partes[0]
        costo  = float(partes[1])
        precio = None if partes[2].strip() == "-" else float(partes[2])
        stock  = int(partes[3])
        tienda = partes[4] if len(partes) > 4 else None
        fecha  = None
        if len(partes) > 5:
            try:
                fecha = datetime.strptime(partes[5].strip(), "%d/%m/%Y").strftime("%
            except ValueError:
                fecha = None
        return nombre, costo, precio, stock, tienda, fecha
    except (ValueError, IndexError):
        return None

def resumen_prenda(nombre, costo, precio, stock, tienda=None, fecha=None) -> str:
    ganancia   = precio - (costo / stock) if stock > 0 else 0
    costo_unit = round(costo / stock, 2) if stock > 0 else 0
    margen     = (ganancia / costo_unit * 100) if costo_unit > 0 else 0
    estado     = calcular_estado(stock)
    partes = [
        "Guardado en Notion!",
        "",
```

```
        f"Prenda: {nombre}",
        f"Costo total: S/{costo:.0f}",
        f"Costo unitario: S/{costo_unit:.2f}",
        f"Precio venta: S/{precio:.0f}",
        f"Stock inicial: {stock} unidades",
        f"Estado: {estado}",
        f"Ganancia por unidad: S/{ganancia:.0f} ({margen:.0f}%)",
    ]
    if tienda:
        partes.append(f"Tienda: {tienda}")
    if fecha:
        fecha_fmt = datetime.strptime(fecha, "%Y-%m-%d").strftime("%d/%m/%Y")
        partes.append(f"Fecha de compra: {fecha_fmt}")
    return "\n".join(partes)

def fetch_tiendas_registradas() -> list:
    """Devuelve lista de tiendas únicas que ya existen en el inventario."""
    prendas = fetch_inventario_completo()
    if not prendas:
        return []
    tiendas = sorted({p.get("tienda","").strip() for p in prendas if p.get("tienda"
    return tiendas

def fetch_inventario_completo():
    url     = f"https://api.notion.com/v1/databases/{NOTION_DATABASE_ID}/query"
    payload = {"sorts": [{"property": "Prenda", "direction": "ascending"}]}
    pages   = []
    prendas = []
    while True:
        r = requests.post(url, headers=NOTION_HEADERS, json=payload)
        if r.status_code != 200:
            return None
        data = r.json()
        for page in data.get("results", []):
            props      = page["properties"]
            titulo     = props.get("Prenda", {}).get("title", [])
            nombre     = titulo[0]["text"]["content"] if titulo else "Sin nombre"
            stock      = props.get("Stock",          {}).get("number", 0) or 0
            stock_ini  = props.get("Stock Inicial",  {}).get("number", 0) or stock
            precio     = props.get("Precio",         {}).get("number", 0) or 0
            costo      = props.get("Costo",          {}).get("number", 0) or 0
            costo_u    = props.get("Costo Unitario", {}).get("number", 0) or 0
            # Fallback: calcular costo_u si el campo está vacío
            if costo_u == 0 and costo > 0 and stock_ini > 0:
                costo_u = round(costo / stock_ini, 2)
            tienda_rt  = props.get("Tienda",         {}).get("rich_text", [])
            tienda     = tienda_rt[0]["text"]["content"] if tienda_rt else ""
            fecha_d    = props.get("Fecha Compra",   {}).get("date") or {}
            fecha      = fecha_d.get("start", "")
            estado     = calcular_estado(stock)
            vendidas   = max(0, stock_ini - stock)
            ganancia_u = precio - costo_u
            margen     = round(ganancia_u / costo_u * 100, 1) if costo_u > 0 else 0
            ingreso_real   = vendidas * precio
            ganancia_real  = vendidas * ganancia_u
            valor_restante = stock * precio
```

```
            inversion_rest = stock * costo_u
            prendas.append({
                "id": page["id"], "nombre": nombre, "stock": stock,
                "stock_ini": stock_ini, "vendidas": vendidas, "precio": precio,
                "costo": costo, "costo_u": costo_u, "ganancia_u": ganancia_u,
                "margen": margen, "ingreso_real": ingreso_real,
                "ganancia_real": ganancia_real, "valor_restante": valor_restante,
                "inversion_rest": inversion_rest, "tienda": tienda, "fecha": fecha,
                "estado": estado,
            })
        if not data.get("has_more"):
            break
        payload["start_cursor"] = data["next_cursor"]
    return prendas

def historial_ventas_prenda(nombre_prenda: str) -> dict:
    if not NOTION_VENTAS_ID:
        return {}
    url = f"https://api.notion.com/v1/databases/{NOTION_VENTAS_ID}/query"
    payload = {
        "filter": {"property": "Prenda", "rich_text": {"contains": nombre_prenda[:
        "sorts": [{"property": "Fecha", "direction": "ascending"}],
        "page_size": 100
    }
    r = requests.post(url, headers=NOTION_HEADERS, json=payload)
    if r.status_code != 200:
        return {}
    resultados = r.json().get("results", [])
    ventas = []
    for page in resultados:
        props = page.get("properties", {})
        try:
            prenda_r = props.get("Prenda", {}).get("rich_text", [])
            nombre_v = prenda_r[0]["text"]["content"] if prenda_r else ""
            if nombre_prenda.lower() not in nombre_v.lower():
                continue
            fecha_d  = props.get("Fecha", {}).get("date") or {}
            fecha    = fecha_d.get("start", "")[:10]
            cantidad = props.get("Cantidad",    {}).get("number") or 0
            precio_r = props.get("Precio real", {}).get("number") or 0
            ganancia = props.get("Ganancia",    {}).get("number") or 0
            cliente  = ""
            cliente_r = props.get("Cliente", {}).get("rich_text", [])
            if cliente_r:
                cliente = cliente_r[0]["text"]["content"]
            ventas.append({"fecha": fecha, "cantidad": cantidad,
                           "precio_real": precio_r, "ganancia": ganancia, "cliente":
        except Exception:
            continue
    total_uds      = sum(v["cantidad"] for v in ventas)
    total_ganancia = sum(v["ganancia"] for v in ventas)
    clientes_unicos = len({v["cliente"] for v in ventas if v["cliente"]})
    return {
        "ventas": ventas, "total_uds": total_uds,
        "total_ganancia": total_ganancia, "clientes_unicos": clientes_unicos,
        "num_transacciones": len(ventas),
```

```
    }

def registrar_venta_notion(nombre_prenda, cantidad, precio_real, costo_u,
                           ganancia, cliente="", fecha_venta=None, descuento=0.0) -> b
    if not NOTION_VENTAS_ID:
        return True
    from datetime import datetime, timezone
    fecha = fecha_venta or datetime.now(timezone.utc).strftime("%Y-%m-%d")
    url  = "https://api.notion.com/v1/pages"
    data = {
        "parent": {"database_id": NOTION_VENTAS_ID},
        "properties": {
            "Venta":         {"title": [{"text": {"content": f"Venta {nombre_prenda}"
            "Fecha":         {"date": {"start": fecha}},
            "Prenda":        {"rich_text": [{"text": {"content": nombre_prenda}}]},
            "Cantidad":      {"number": cantidad},
            "Precio real":   {"number": precio_real},
            "Costo unitario":{"number": round(costo_u, 2)},
            "Descuento":     {"number": round(descuento, 2)},
            "Ganancia":      {"number": round(ganancia, 2)},
            "Cliente":       {"rich_text": [{"text": {"content": cliente}}]},
        }
    }
    r = requests.post(url, headers=NOTION_HEADERS, json=data)
    if r.status_code not in (200, 201):
        logger.error(f"Error Notion Ventas {r.status_code}: {r.text[:300]}")
        return False
    return True

def eliminar_venta_notion(page_id: str) -> bool:
    """Elimina un registro de venta de Notion y restaura el stock de la prenda."""
    # Primero obtener datos de la venta para restaurar stock
    r = requests.get(f"https://api.notion.com/v1/pages/{page_id}", headers=NOTION_H
    if r.status_code != 200:
        return False
    props    = r.json().get("properties", {})
    prenda_n = props.get("Prenda", {}).get("rich_text", [])
    cantidad = props.get("Cantidad", {}).get("number", 0) or 0
    nombre   = prenda_n[0]["text"]["content"] if prenda_n else ""
    # Archivar (eliminar) la venta en Notion
    r2 = requests.patch(
        f"https://api.notion.com/v1/pages/{page_id}",
        headers=NOTION_HEADERS,
        json={"archived": True}
    )
    if r2.status_code != 200:
        return False
    # Restaurar stock en inventario
    if nombre and cantidad > 0:
        prendas = buscar_prendas_notion(nombre)
        if prendas:
            stock_actual = prendas[0]["stock"]
            actualizar_prenda_notion(prendas[0]["id"], {"Stock": {"number": stock_ac
    return True

def buscar_ventas_notion(termino: str) -> list:
```

```
    """Busca ventas recientes en la BD de ventas por nombre de prenda."""
    if not NOTION_VENTAS_ID:
        return []
    url     = f"https://api.notion.com/v1/databases/{NOTION_VENTAS_ID}/query"
    payload = {
        "sorts": [{"property": "Fecha", "direction": "descending"}],
        "page_size": 20,
    }
    r = requests.post(url, headers=NOTION_HEADERS, json=payload)
    if r.status_code != 200:
        return []
    resultados = []
    for page in r.json().get("results", []):
        props    = page["properties"]
        prenda_n = props.get("Prenda", {}).get("rich_text", [])
        nombre   = prenda_n[0]["text"]["content"] if prenda_n else ""
        if termino.lower() in nombre.lower() or not termino:
            cantidad = props.get("Cantidad", {}).get("number", 0) or 0
            precio   = props.get("Precio real", {}).get("number", 0) or 0
            fecha_d  = props.get("Fecha", {}).get("date") or {}
            fecha    = fecha_d.get("start", "")
            resultados.append({
                "id":       page["id"],
                "nombre":   nombre,
                "cantidad": cantidad,
                "precio":   precio,
                "fecha":    fecha,
                "label":    f"{nombre} | {cantidad}ud x S/{precio:.0f} | {fecha}",
            })
    return resultados

def _formato_stock(prenda: dict) -> str:
    estado      = calcular_estado(prenda["stock"])
    stock_ini   = prenda.get("stock_inicial", 0) or prenda["stock"]
    vendidas    = max(0, stock_ini - prenda["stock"])
    pct_vendido = round(vendidas / stock_ini * 100) if stock_ini > 0 else 0
    costo_u     = prenda.get("costo_u", 0) or 0
    costo_total = prenda.get("costo", 0) or 0
    ganancia_u  = prenda["precio"] - costo_u if costo_u > 0 else 0
    margen      = round(ganancia_u / costo_u * 100) if costo_u > 0 else 0
    tienda      = prenda.get("tienda", "") or ""
    fecha       = prenda.get("fecha", "") or ""
    lineas = [
        f"▯ {prenda['nombre']}", "",
        f"Stock actual:    {prenda['stock']} uds",
        f"Stock inicial:   {stock_ini} uds",
        f"Vendidas:        {vendidas} uds ({pct_vendido}%)",
        f"Estado:          {estado}", "",
        f"Precio venta:    S/{prenda['precio']:.0f}",
        f"Costo unitario:  S/{costo_u:.2f}",
        f"Costo total:     S/{costo_total:.0f}",
        f"Ganancia/ud:     S/{ganancia_u:.0f} ({margen}%)",
    ]
    if tienda:
        lineas.append(f"Tienda:         {tienda}")
    if fecha:
```

```
        lineas.append(f"Fecha compra:    {fecha}")
    hist = historial_ventas_prenda(prenda["nombre"])
    if hist and hist.get("num_transacciones", 0) > 0:
        lineas += ["", "─── Historial de ventas ───",
                   f"Transacciones:   {hist['num_transacciones']}",
                   f"Uds vendidas:    {hist['total_uds']}"]
        if hist.get("clientes_unicos", 0) > 0:
            lineas.append(f"Clientes únicos: {hist['clientes_unicos']}")
            nombres = sorted({v["cliente"] for v in hist["ventas"] if v["cliente"]})
            if nombres:
                lineas.append("Compradoras:     " + ", ".join(nombres))
        lineas.append(f"Ganancia total:  S/{hist['total_ganancia']:.0f}")
        if hist['total_uds'] > 0:
            gan_prom = hist['total_ganancia'] / hist['total_uds']
            lineas.append(f"Ganancia/ud:     S/{gan_prom:.0f}")
        lineas.append("")
        for v in hist["ventas"]:
            fecha_fmt  = v["fecha"][5:7] + "/" + v["fecha"][8:10] + "/" + v["fecha"][:
            cliente_str = f" — {v['cliente']}" if v["cliente"] else ""
            lineas.append(f"  {fecha_fmt}  {v['cantidad']} ud  S/{v['precio_real']:.0
    else:
        lineas += ["", "Sin ventas registradas aún."]
    return "\n".join(lineas)

def _texto_agotados() -> str:
    url     = f"https://api.notion.com/v1/databases/{NOTION_DATABASE_ID}/query"
    payload = {"page_size": 50}
    r = requests.post(url, headers=NOTION_HEADERS, json=payload)
    if r.status_code != 200:
        return f"Error al consultar Notion ({r.status_code})."
    resultados = r.json().get("results", [])
    agotadas = []
    for page in resultados:
        props  = page["properties"]
        stock  = props.get("Stock",  {}).get("number")
        estado = props.get("Estado", {}).get("select") or {}
        nombre_data = props.get("Prenda", {}).get("title", [])
        nombre = nombre_data[0]["text"]["content"] if nombre_data else "Sin nombre"
        precio = props.get("Precio", {}).get("number", 0) or 0
        es_agotada = (stock is None or stock == 0 or estado.get("name") == "Agotado"
        if es_agotada:
            agotadas.append({"nombre": nombre, "precio": precio})
    if not agotadas:
        return "Todo bien! No hay prendas agotadas ahora mismo."
    lineas = [f"Prendas agotadas ({len(agotadas)}):\n"]
    for p in agotadas:
        lineas.append(f"- {p['nombre']} - S/{p['precio']:.0f}")
    return "\n".join(lineas)

# ═══════════════════════════════════════════════════════════
# GRÁFICOS
# ═══════════════════════════════════════════════════════════
def generar_grafico_stock() -> io.BytesIO | None:
    prendas = fetch_inventario_completo()
    if not prendas:
        return None
```

```
    prendas = sorted(prendas, key=lambda p: p["stock"], reverse=True)
    nombres = [p["nombre"][:22] + "…" if len(p["nombre"]) > 22 else p["nombre"] for
    stocks  = [p["stock"] for p in prendas]
    estados = [calcular_estado(p["stock"]) for p in prendas]
    colores_map = {"Disponible": "#2e7d32", "Stock limitado": "#f9a825", "Agotado":
    colores = [colores_map.get(e, "#999") for e in estados]
    alto = max(4, len(prendas) * 0.45)
    fig, ax = plt.subplots(figsize=(9, alto))
    fig.patch.set_facecolor("#fafafa")
    ax.set_facecolor("#fafafa")
    bars = ax.barh(nombres, stocks, color=colores, edgecolor="white", height=0.65)
    for bar, val in zip(bars, stocks):
        x_pos = bar.get_width()
        ax.text(x_pos + 0.15, bar.get_y() + bar.get_height() / 2,
                str(val), va="center", ha="left", fontsize=9, color="#333", fontweigh
    ax.set_xlabel("Unidades en stock", fontsize=10, color="#555")
    ax.set_title("Stock actual por prenda — Maricuchis Store", fontsize=13, fontweig
    ax.invert_yaxis()
    ax.xaxis.grid(True, linestyle="--", alpha=0.4)
    ax.set_axisbelow(True)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    leyenda = [
        mpatches.Patch(color="#2e7d32", label="Disponible"),
        mpatches.Patch(color="#f9a825", label="Stock limitado (1-3)"),
        mpatches.Patch(color="#c62828", label="Agotado"),
    ]
    ax.legend(handles=leyenda, loc="lower right", fontsize=8, framealpha=0.6)
    plt.tight_layout()
    buf = io.BytesIO()
    plt.savefig(buf, format="png", dpi=130, bbox_inches="tight")
    plt.close(fig)
    buf.seek(0)
    return buf

# ============================================================
# IA — GEMINI FLASH INTEGRADO
# ============================================================

SYSTEM_PROMPT = """Eres la asistente de Maricuchis Store, una tienda de ropa al por
Tu nombre es Maricuchis IA. La dueña se llama Maritza.

PUEDES HACER LO SIGUIENTE (responde SIEMPRE en JSON con la estructura indicada):

1. CONSULTAR datos del inventario o ventas → ya tienes los datos en el contexto
2. CALCULAR: márgenes, ganancias, ROI, proyecciones, precios sugeridos
3. ACTUALIZAR precio de una prenda → {"accion": "actualizar_precio", "prenda": "nom
4. ACTUALIZAR stock de una prenda → {"accion": "actualizar_stock", "prenda": "nombre
5. REGISTRAR una venta → {"accion": "registrar_venta", "prenda": "nombre", "cantida
6. CREAR prenda nueva → {"accion": "crear_prenda", "nombre": "...", "costo": 60, "
7. REDACTAR texto para WhatsApp → {"accion": "texto_whatsapp", "texto": "...mensaje
8. RESPONDER preguntas → {"accion": "responder", "texto": "...tu respuesta..."}
9. SUGERIR precio → {"accion": "responder", "texto": "...recomendación con justifi

ACCIONES EXTRA:
7. GENERAR gráfico de stock → {"accion": "generar_grafico", "tipo": "stock"}
```

```
8. CREAR prenda con foto (cuando recibes imagen) → {"accion": "crear_prenda_con_foto
9. MOSTRAR prenda con foto → {"accion": "mostrar_prenda", "prenda": "nombre exacto
   Usa esta acción cuando: te pidan VER una prenda, consulten su precio/stock/detalle
   SIEMPRE incluye el campo "texto" con los datos de la prenda (precio, stock, estad

REGLAS:
- Responde SIEMPRE en JSON válido, sin texto adicional fuera del JSON
- Para acciones sobre prendas, usa el nombre más parecido que encuentres en el inve
- MEMORIA: el historial de la conversación se incluye como turnos previos. Úsalo par
- Si no entiendes la solicitud, usa {"accion": "responder", "texto": "No entendí. ¿
- Si algo está fuera de tu alcance (no es del negocio), usa {"accion": "responder",
- Usa soles peruanos (S/) siempre
- Sé breve y directa en las respuestas de texto
- Si te piden texto para WhatsApp, hazlo atractivo con emojis, menciona precio y dis
- Si recibes una imagen, descríbela y úsala para registrar la prenda con accion crea

def llamar_gemini(mensaje_usuario: str, contexto_inventario: str,
                  historial: list = None, imagen_bytes: bytes = None) -> dict:
    """Llama a Gemini 2.5 Flash con historial multi-turn y soporte de imagen."""
    if not GEMINI_API_KEY:
        return {"accion": "responder", "texto": "GEMINI_API_KEY no configurada en Ra

    url = (
        "https://generativelanguage.googleapis.com/v1beta/models/"
        "gemini-2.5-flash:generateContent?key=" + GEMINI_API_KEY
    )

    # Turno inicial: sistema + inventario
    sistema_texto = SYSTEM_PROMPT + "\n\nINVENTARIO ACTUAL:\n" + contexto_inventario
    contents = [
        {"role": "user",  "parts": [{"text": sistema_texto}]},
        {"role": "model", "parts": [{"text": '{"accion":"responder","texto":"Enten
    ]

    # Historial previo de la sesión
    for turno in (historial or []):
        contents.append({"role": "user",  "parts": [{"text": turno["usuario"]}]})
        contents.append({"role": "model", "parts": [{"text": turno["ia_raw"]}]})

    # Mensaje actual (con imagen opcional)
    partes = []
    if imagen_bytes:
        img_b64 = base64.b64encode(imagen_bytes).decode("utf-8")
        partes.append({"inline_data": {"mime_type": "image/jpeg", "data": img_b64}
    partes.append({"text": mensaje_usuario})
    contents.append({"role": "user", "parts": partes})

    payload = {
        "contents": contents,
        "generationConfig": {"temperature": 0.3, "maxOutputTokens": 1024},
    }

    try:
        r = requests.post(url, json=payload, timeout=30)
        if r.status_code != 200:
            logger.error("Error Gemini %s: %s", r.status_code, r.text[:300])
```

```
            return {"accion": "responder", "texto": "Error al contactar la IA. Inten

        respuesta_raw = r.json()["candidates"][0]["content"]["parts"][0]["text"].stri
        if respuesta_raw.startswith("```"):
            respuesta_raw = respuesta_raw.split("```")[1]
            if respuesta_raw.startswith("json"):
                respuesta_raw = respuesta_raw[4:]
        resultado = json.loads(respuesta_raw)
        resultado["_raw"] = respuesta_raw
        return resultado

    except json.JSONDecodeError:
        return {"accion": "responder", "texto": respuesta_raw, "_raw": respuesta_ra
    except Exception as e:
        logger.error("Error Gemini: %s", e)
        return {"accion": "responder", "texto": "Hubo un error con la IA. Intenta de

def construir_contexto_inventario() -> str:
    """Construye un resumen del inventario para pasarle a Gemini como contexto."""
    prendas = fetch_inventario_completo()
    if not prendas:
        return "Inventario vacío."

    lineas = [f"Total prendas: {len(prendas)}\n"]
    for p in prendas:
        lineas.append(
            f"- {p['nombre']} | stock: {p['stock']} | precio: S/{p['precio']} | "
            f"costo_u: S/{p['costo_u']:.2f} | vendidas: {p['vendidas']} | "
            f"ganancia_u: S/{p['ganancia_u']:.2f} | tienda: {p['tienda']} | estado: {p
        )

    total_invertido  = sum(p["costo"] for p in prendas)
    total_ganancia   = sum(p["ganancia_real"] for p in prendas)
    total_vendidas   = sum(p["vendidas"] for p in prendas)
    ganancia_pot     = sum(p["stock"] * p["ganancia_u"] for p in prendas)

    lineas.append(f"\nRESUMEN FINANCIERO:")
    lineas.append(f"- Total invertido: S/{total_invertido:.0f}")
    lineas.append(f"- Ganancia realizada: S/{total_ganancia:.0f}")
    lineas.append(f"- Unidades vendidas: {total_vendidas}")
    lineas.append(f"- Ganancia potencial (stock restante): S/{ganancia_pot:.0f}")

    return "\n".join(lineas)

async def ejecutar_accion_ia(accion_dict: dict, update: Update) -> str:
    """Ejecuta la acción que Gemini indicó y retorna el mensaje de respuesta."""
    accion = accion_dict.get("accion", "responder")

    # --- Solo responder ---
    if accion == "responder":
        return accion_dict.get("texto", "No tengo respuesta para eso.")

    # --- Texto para WhatsApp ---
    if accion == "texto_whatsapp":
        texto = accion_dict.get("texto", "")
        return f"□ Texto listo para WhatsApp:\n\n{texto}"
```

```
    # --- Actualizar precio ---
    if accion == "actualizar_precio":
        nombre  = accion_dict.get("prenda", "")
        precio  = accion_dict.get("precio")
        if not nombre or precio is None:
            return "Falta el nombre de la prenda o el precio."
        prendas = buscar_prendas_notion(nombre)
        if not prendas:
            return f"No encontré la prenda '{nombre}' en el inventario."
        prenda = prendas[0]
        exito  = actualizar_prenda_notion(prenda["id"], {"Precio": {"number": float
        if exito:
            return f"✅ Precio de '{prenda['nombre']}' actualizado a S/{precio:.0f}"
        return f"❌ Error al actualizar el precio de '{prenda['nombre']}'."

    # --- Actualizar stock ---
    if accion == "actualizar_stock":
        nombre = accion_dict.get("prenda", "")
        stock  = accion_dict.get("stock")
        if not nombre or stock is None:
            return "Falta el nombre de la prenda o el stock."
        prendas = buscar_prendas_notion(nombre)
        if not prendas:
            return f"No encontré la prenda '{nombre}' en el inventario."
        prenda = prendas[0]
        exito  = actualizar_prenda_notion(prenda["id"], {"Stock": {"number": int(sto
        if exito:
            return f"✅ Stock de '{prenda['nombre']}' actualizado a {stock} uds"
        return f"❌ Error al actualizar el stock de '{prenda['nombre']}'."

    # --- Registrar venta ---
    if accion == "registrar_venta":
        nombre   = accion_dict.get("prenda", "")
        cantidad = int(accion_dict.get("cantidad", 1))
        precio_v = float(accion_dict.get("precio", 0))
        cliente  = accion_dict.get("cliente", "")
        if not nombre or precio_v <= 0:
            return "Falta el nombre de la prenda o el precio de venta."
        prendas = buscar_prendas_notion(nombre)
        if not prendas:
            return f"No encontré la prenda '{nombre}'."
        prenda = prendas[0]
        if prenda["stock"] < cantidad:
            return f"❌ Solo hay {prenda['stock']} uds de '{prenda['nombre']}'. No se
        costo_u  = prenda["costo_u"]
        descuento_ia = float(accion_dict.get("descuento", 0) or 0)
        ganancia = (precio_v - descuento_ia - costo_u) * cantidad
        nuevo_stock = prenda["stock"] - cantidad
        # Descontar stock
        actualizar_prenda_notion(prenda["id"], {"Stock": {"number": nuevo_stock}})
        # Registrar en BD ventas
        registrar_venta_notion(prenda["nombre"], cantidad, precio_v, costo_u, ganan
        return (
            f"✅ Venta registrada!\n"
            f"Prenda: {prenda['nombre']}\n"
```

```
        f"Cantidad: {cantidad} uds\n"
        f"Precio: S/{precio_v:.0f}\n"
        f"Ganancia: S/{ganancia:.0f}\n"
        f"Stock restante: {nuevo_stock} uds"
    )

# --- Crear prenda ---
if accion == "crear_prenda":
    nombre = accion_dict.get("nombre", "")
    costo  = float(accion_dict.get("costo", 0))
    precio = float(accion_dict.get("precio", 0))
    stock  = int(accion_dict.get("stock", 0))
    tienda = accion_dict.get("tienda", None)
    if not nombre or costo <= 0 or precio <= 0 or stock <= 0:
        return "Faltan datos para crear la prenda (nombre, costo, precio, stock)
    exito = crear_prenda_notion(nombre, costo, precio, stock, tienda=tienda)
    if exito:
        costo_u  = round(costo / stock, 2)
        ganancia = precio - costo_u
        return (
            f"✅ Prenda creada!\n"
            f"Nombre: {nombre}\n"
            f"Costo unitario: S/{costo_u:.2f}\n"
            f"Precio: S/{precio:.0f}\n"
            f"Stock: {stock} uds\n"
            f"Ganancia/ud: S/{ganancia:.0f}"
        )
    return f"❌ Error al crear la prenda '{nombre}'."

# --- Mostrar prenda con foto ---
if accion == "mostrar_prenda":
    nombre = accion_dict.get("prenda", "")
    texto  = accion_dict.get("texto", "")
    if not nombre:
        return texto or "No se especificó la prenda."
    prendas = buscar_prendas_notion(nombre)
    if not prendas:
        return texto or "No encontré esa prenda en el inventario."
    prenda   = prendas[0]
    foto_url = obtener_foto_url(prenda["id"])
    # Devolver señal especial con datos para que ia_procesar_mensaje envíe la fo
    return "__MOSTRAR_FOTO__" + "|||" + (foto_url or "") + "|||" + (texto or nomb

# --- Generar grafico ---
if accion == "generar_grafico":
    return "__GENERAR_GRAFICO__"

# --- Crear prenda con foto (IA recibio imagen) ---
if accion == "crear_prenda_con_foto":
    nombre   = accion_dict.get("nombre", "")
    costo    = float(accion_dict.get("costo", 0))
    precio   = float(accion_dict.get("precio", 0))
    stock    = int(accion_dict.get("stock", 0))
    tienda   = accion_dict.get("tienda", None)
    foto_url = accion_dict.get("_foto_url", None)
    if not nombre or costo <= 0 or precio <= 0 or stock <= 0:
```

```
            return "Faltan datos para crear la prenda (nombre, costo, precio, stock)
        exito = crear_prenda_notion(nombre, costo, precio, stock, foto_url=foto_url
        if exito:
            costo_u = round(costo / stock, 2)
            ganancia_u = precio - costo_u
            con_foto = " con foto" if foto_url else " (sin foto)"
            return (
                "Prenda creada" + con_foto + "!\n"
                + "Nombre: " + nombre + "\n"
                + "Costo unitario: S/" + f"{costo_u:.2f}" + "\n"
                + "Precio: S/" + f"{precio:.0f}" + "\n"
                + "Stock: " + str(stock) + " uds\n"
                + "Ganancia/ud: S/" + f"{ganancia_u:.0f}"
            )
        return "Error al crear la prenda " + nombre + "."

    return "No entendi la accion. Intenta de nuevo."

# ==========================================================
# HELPERS — INTERFAZ
# ==========================================================
def teclado_lista_prendas(prendas: list, accion: str) -> InlineKeyboardMarkup:
    botones = []
    for p in prendas:
        label = f"{p['nombre']} (stock: {p['stock']})"
        botones.append([InlineKeyboardButton(label, callback_data=f"{accion}:{p['id'
    botones.append([InlineKeyboardButton("Cancelar", callback_data="cancelar")])
    return InlineKeyboardMarkup(botones)

def teclado_menu_principal() -> InlineKeyboardMarkup:
    return InlineKeyboardMarkup([
        [
            InlineKeyboardButton("□ Nueva prenda",    callback_data="menu_nueva_guiac
            InlineKeyboardButton("□ Sin foto",        callback_data="menu_sinfoto"),
        ],
        [
            InlineKeyboardButton("□ Adjuntar foto",  callback_data="menu_adjfoto"),
            InlineKeyboardButton("□ Registrar venta", callback_data="menu_venta"),
        ],
        [
            InlineKeyboardButton("□ Consultar prenda",callback_data="menu_stock"),
            InlineKeyboardButton("□ Ver agotados",    callback_data="menu_agotados")
        ],
        [
            InlineKeyboardButton("✏ Actualizar / Eliminar prenda", callback_data="me
        ],
        [
            InlineKeyboardButton("□ Eliminar venta", callback_data="menu_eliminar_ve
        ],
        [
            InlineKeyboardButton("□ Ver foto",        callback_data="menu_verfoto"),
            InlineKeyboardButton("□ Ver inventario",  callback_data="menu_inventario"
        ],
        [InlineKeyboardButton("□ Gráfico stock",      callback_data="menu_graficostoc
        [
            InlineKeyboardButton("□ Ganancias",         callback_data="menu_ganancias
```

```
            InlineKeyboardButton("⚖ Comparar prendas",   callback_data="menu_compar
        ],
        [InlineKeyboardButton("□ Preguntarle a la IA",   callback_data="menu_ia")],
        [InlineKeyboardButton("? Ayuda",                 callback_data="menu_ayuda")]
    ])

def _texto_ayuda() -> str:
    return (
        "Guia de uso - Maricuchis Store Bot v4.24\n\n"
        "REGISTRAR PRENDAS\n"
        "/sinfoto - Registrar prenda sin foto (paso a paso)\n"
        "/adjfoto - Adjuntar foto a prenda ya registrada\n\n"
        "VENTAS Y STOCK\n"
        "/venta   - Registrar venta y descontar stock\n"
        "/prenda  - Consultar detalle de una prenda\n"
        "/agotados - Ver todas las prendas sin stock\n\n"
        "VER Y CONSULTAS\n"
        "/verfoto    - Ver la foto de una prenda\n"
        "/inventario - Ver todas las prendas del inventario\n\n"
        "FINANZAS\n"
        "/resumen    - Resumen financiero completo\n"
        "/comparar   - Comparar 2 o mas prendas\n"
        "/pormargen  - Ranking por rentabilidad\n"
        "/portienda  - Resumen agrupado por tienda\n\n"
        "□ IA INTEGRADA\n"
        "/ia - Activar modo conversacional con la IA\n"
        "      Ejemplos de lo que puedes decirle:\n"
        "      'Actualiza el precio de la blusa a S/35'\n"
        "      'Escríbeme texto para WhatsApp de la chompa'\n"
        "      '¿Cuáles prendas debo reponer?'\n"
        "      'Registra que vendí 2 blusas a S/30'\n"
        "      'Crea una prenda: blusa blanca, costo 60, precio 90, stock 12'\n"
        "      '¿Cuánto gané esta semana?'\n\n"
        "EDITAR INVENTARIO\n"
        "/actualizar - Editar datos de una prenda\n"
        "/eliminar   - Eliminar una prenda\n\n"
        "OTROS\n"
        "/menu     - Ver el menu con botones\n"
        "/chatid   - Ver tu Chat ID\n"
        "/cancelar - Cancelar lo que estes haciendo\n"
        "/ayuda    - Mostrar esta guia"
    )

# ============================================================
# HANDLERS — COMANDOS GENERALES
# ============================================================
async def _reply(update: Update, texto: str, **kwargs):
    if update.message:
        await update.message.reply_text(texto, **kwargs)
    elif update.callback_query:
        await update.callback_query.message.reply_text(texto, **kwargs)

async def cmd_start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "Maricuchis Store - Bot v3.0\n\nHola Maritza! ¿Qué quieres hacer hoy?\n\n"
        "Novedad: usa /ia para hablarle directamente a la IA □",
```

```
        reply_markup=teclado_menu_principal()
    )

async def cmd_menu(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Menu principal:", reply_markup=teclado_menu_pr

async def cmd_ayuda(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(_texto_ayuda())

async def cmd_cancelar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    tenia_sesion_ia = bool(context.user_data.get("historial_ia"))
    context.user_data.clear()
    if tenia_sesion_ia:
        await _reply(update, "Modo IA cerrado. Memoria borrada. ✅\n\nUsa /ia para i
    else:
        await _reply(update, "Operacion cancelada.")
    # Siempre mostrar el menú al cancelar
    await _mostrar_menu(update, context)
    return ConversationHandler.END

async def _mostrar_menu(update, context):
    """Muestra el menú principal desde cualquier contexto."""
    teclado = teclado_menu_principal()
    texto   = "Menú principal:"
    if update.callback_query:
        await update.callback_query.message.reply_text(texto, reply_markup=teclado)
    elif update.message:
        await update.message.reply_text(texto, reply_markup=teclado)

async def fallback_menu_inicio(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Fallback para el botón Volver al menú dentro de cualquier ConversationHandler
    query = update.callback_query
    if query:
        await query.answer()
        context.user_data.clear()
        await query.message.reply_text("Menú principal:", reply_markup=teclado_menu
    return ConversationHandler.END

async def cmd_chatid(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(f"Tu Chat ID es: {update.effective_chat.id}")

# ============================================================
# COMANDO /ia — MODO CONVERSACIONAL CON GEMINI
# ============================================================
IA_ESPERANDO = 200

async def cmd_ia(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if not GEMINI_API_KEY:
        await _reply(update,
            "⚠ La IA no está configurada aún.\n\n"
            "Para activarla, agrega GEMINI_API_KEY en las variables de Railway.\n"
            "Obtén tu clave gratis en: aistudio.google.com"
        )
        return ConversationHandler.END
    await _reply(update,
        "□ Modo IA activado!\n\n"
```

```
        "Escríbeme lo que necesitas en lenguaje natural. Ejemplos:\n\n"
        "• 'Actualiza el precio de la blusa floral a S/35'\n"
        "• 'Escríbeme texto para WhatsApp de la chompa negra'\n"
        "• '¿Qué prendas me conviene reponer?'\n"
        "• 'Registra que vendí 2 pantalones a S/40'\n"
        "• '¿Cuánto he ganado esta semana?'\n"
        "• 'Sugiereme precio para prenda que me costó S/96 la docena'\n\n"
        "(Escribe /cancelar para salir del modo IA)"
    )
    return IA_ESPERANDO

async def ia_procesar_mensaje(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg = update.message

    # Detectar foto o texto
    imagen_bytes = None
    foto_url_subida = None
    if msg.photo:
        await msg.reply_text("Procesando imagen...")
        foto    = msg.photo[-1]
        archivo = await context.bot.get_file(foto.file_id)
        img_ba  = await archivo.download_as_bytearray()
        imagen_bytes    = bytes(img_ba)
        foto_url_subida = subir_imagen(imagen_bytes)
        mensaje = msg.caption.strip() if msg.caption else "Registra esta prenda en e
    elif msg.text:
        mensaje = msg.text.strip()
    else:
        await msg.reply_text("Solo entiendo texto o fotos en modo IA.")
        return IA_ESPERANDO

    await msg.reply_text("Pensando...")

    historial = context.user_data.get("historial_ia", [])
    contexto  = construir_contexto_inventario()
    accion_dict = llamar_gemini(mensaje, contexto, historial=historial, imagen_byte

    # Pasar URL de foto si la IA quiere crear prenda con foto
    if accion_dict.get("accion") == "crear_prenda_con_foto" and foto_url_subida:
        accion_dict["_foto_url"] = foto_url_subida

    respuesta = await ejecutar_accion_ia(accion_dict, update)

    if respuesta == "__GENERAR_GRAFICO__":
        await msg.reply_text("Generando grafico...")
        buf = generar_grafico_stock()
        if buf:
            await msg.reply_photo(photo=buf, caption="Stock actual — Maricuchis Store
        else:
            await msg.reply_text("No se pudo generar el grafico.")
    elif respuesta.startswith("__MOSTRAR_FOTO__"):
        partes_foto = respuesta.split("|||")
        foto_url  = partes_foto[1] if len(partes_foto) > 1 else ""
        texto_ia  = partes_foto[2] if len(partes_foto) > 2 else ""
        if foto_url:
            try:
```

```
                await msg.reply_photo(photo=foto_url, caption=texto_ia)
            except Exception:
                await msg.reply_text(texto_ia + "\n\n(No se pudo cargar la foto)")
        else:
            await msg.reply_text(texto_ia + "\n\n(Esta prenda no tiene foto aún)")
    else:
        await msg.reply_text(respuesta)

    # Guardar turno en historial (max 10 turnos)
    raw = accion_dict.get("_raw", json.dumps(accion_dict))
    historial.append({"usuario": mensaje, "ia_raw": raw})
    context.user_data["historial_ia"] = historial[-10:]

    return IA_ESPERANDO

# ============================================================
# COMANDO /nueva
# ============================================================
async def cmd_nueva(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update,
        "Nueva prenda con foto\n\n"
        "Envía la foto con pie de foto así:\n"
        "  Nombre, costo, precio, stock\n\n"
        "Opcionales al final:\n"
        "  , tienda, DD/MM/YYYY\n\n"
        "Ejemplo:\n"
        "  Chompa azul, 96, 130, 12, Gamarra, 01/05/2026\n\n"
        "O usa /sinfoto si no tienes foto aún."
    )

# ============================================================
# COMANDO /limitadas
# ============================================================
async def cmd_limitadas(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Buscando prendas con stock limitado...")
    prendas = fetch_inventario_completo()
    if prendas is None:
        await _reply(update, "Error al consultar Notion.")
        return
    limitadas = [p for p in prendas if 1 <= p["stock"] <= 3]
    if not limitadas:
        await _reply(update, "No hay prendas con stock limitado en este momento.")
        return
    limitadas.sort(key=lambda p: p["stock"])
    lineas = [f"Prendas con stock bajo ({len(limitadas)}):\n"]
    for p in limitadas:
        lineas.append(f"  {p['nombre']}\n  Stock: {p['stock']} uds | Precio: S/{p['pr
    await _reply(update, "\n".join(lineas))

# ============================================================
# COMANDO /ganancia
# ============================================================
async def cmd_ganancia(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Calculando ganancias...")
    prendas = fetch_inventario_completo()
    if prendas is None:
```

```
        await _reply(update, "Error al consultar Notion.")
        return
    if not prendas:
        await _reply(update, "El inventario está vacío.")
        return
    total_vendidas  = sum(p["vendidas"] for p in prendas)
    ingreso_total   = sum(p["ingreso_real"] for p in prendas)
    ganancia_total  = sum(p["ganancia_real"] for p in prendas)
    ganancia_pot    = sum(p["stock"] * p["ganancia_u"] for p in prendas)
    total_invertido = sum(p["costo"] for p in prendas)
    roi = round(ganancia_total / total_invertido * 100, 1) if total_invertido > 0 els
    top3 = sorted(prendas, key=lambda p: p["ganancia_real"], reverse=True)[:3]
    lineas = [
        "Resumen de ganancias\n",
        f"Unidades vendidas:  {total_vendidas} uds",
        f"Ingresos totales:   S/{ingreso_total:.0f}",
        f"Ganancia realizada: S/{ganancia_total:.0f}  (ROI {roi}%)",
        f"Ganancia potencial: S/{ganancia_pot:.0f}  (stock restante)",
        "", "Top 3 más rentables:",
    ]
    for i, p in enumerate(top3, 1):
        lineas.append(f"  {i}. {p['nombre']} — S/{p['ganancia_real']:.0f}")
    await _reply(update, "\n".join(lineas))

async def cmd_agotados(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Buscando prendas agotadas...")
    await update.message.reply_text(_texto_agotados())

# ============================================================
# HANDLER — MENU (botones)
# ============================================================
async def manejar_menu(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query  = update.callback_query
    await query.answer()
    accion = query.data
    await query.edit_message_reply_markup(reply_markup=None)
    if accion == "menu_nueva":
        await query.message.reply_text(
            "Nueva prenda con foto\n\nEnvia la foto con pie de foto asi:\n"
            "  Nombre, costo, precio, stock\n\nOpcionales al final:\n"
            "  , tienda, DD/MM/YYYY\n\nEjemplo:\n"
            "  Chompa azul, 96, 130, 12, Gamarra, 01/05/2026\n\n(Cancela con /cancelar
        )
    elif accion == "menu_sinfoto":
        await cmd_sin_foto(update, context)
    elif accion == "menu_adjfoto":
        await cmd_adj_foto(update, context)
    elif accion == "menu_venta":
        await cmd_vendi(update, context)
    elif accion == "menu_stock":
        await cmd_stock(update, context)
    elif accion == "menu_agotados":
        await query.message.reply_text("Buscando prendas agotadas...")
        await query.message.reply_text(_texto_agotados())
    elif accion == "menu_eliminar_venta":
        await cmd_eliminar_venta(update, context)
```

```
    elif accion == "menu_verfoto":
        await cmd_ver_foto(update, context)
    elif accion == "menu_inventario":
        await cmd_inventario(update, context)
    elif accion == "menu_resumen":
        await cmd_resumen(update, context)
    elif accion == "menu_pormargen":
        await cmd_por_margen(update, context)
    elif accion == "menu_portienda":
        await cmd_por_tienda(update, context)
    elif accion == "menu_ganancias":
        await cmd_ganancias_menu(update, context)
    elif accion == "fin_resumen":
        await cmd_resumen(update, context)
    elif accion == "fin_pormargen":
        await cmd_por_margen(update, context)
    elif accion == "fin_portienda":
        await cmd_por_tienda(update, context)
    elif accion == "fin_porfecha":
        await cmd_ganancias_fecha_menu(update, context)
    elif accion in ("fin_fecha_hoy", "fin_fecha_ayer", "fin_fecha_semana", "fin_fecha
        await cmd_ganancias_por_fecha(update, context)
    elif accion == "menu_graficostock":
        await cmd_grafico_stock(update, context)
    elif accion == "menu_comparar":
        await cmd_comparar(update, context)
    elif accion == "menu_inicio":
        await cmd_menu(update, context)
    elif accion == "menu_nueva_guiado":
        await cmd_nueva_prenda(update, context)
    elif accion == "menu_ia":
        await cmd_ia(update, context)
    elif accion == "menu_ayuda":
        await query.message.reply_text(_texto_ayuda())

# ============================================================
# HANDLER — FOTO NUEVA CON CAPTION
# ============================================================
async def recibir_foto_nueva(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg = update.message
    if not msg.caption:
        await msg.reply_text(
            "Falta el texto.\n\nReenvía la foto con pie de foto:\n"
            "  Nombre, costo, precio, stock\n\nEjemplo: Chompa azul, 96, 130, 12\n\n
            "(Manten presionada la foto antes de enviar para agregar texto)"
        )
        return
    datos = parsear_caption(msg.caption)
    if not datos:
        await msg.reply_text(
            "Formato incorrecto.\n\nRevisa que el texto tenga:\n"
            "  Nombre, costo, precio, stock\n\nEjemplo: Chompa azul, 96, 130, 12\n\n
            "Escribe /ayuda si necesitas ayuda."
        )
        return
    nombre, costo, precio, stock, tienda, fecha = datos
```

```
    if precio is None:
        costo_unit = costo / stock if stock > 0 else costo
        precio = calcular_precio_sugerido(costo_unit)
        precio_auto = True
    else:
        precio_auto = False
    if costo <= 0 or precio <= 0 or stock <= 0:
        await msg.reply_text("El costo, precio y stock deben ser mayores a 0. Revisa
        return
    aviso_precio = f"\n\n□ Precio calculado automáticamente: S/{precio:.0f}" if pre
    await msg.reply_text("Subiendo foto y guardando en inventario...")
    foto      = msg.photo[-1]
    archivo   = await context.bot.get_file(foto.file_id)
    img_bytes = await archivo.download_as_bytearray()
    foto_url  = subir_imagen(bytes(img_bytes))
    exito = crear_prenda_notion(nombre, costo, precio, stock, foto_url, tienda, fec
    if exito:
        await msg.reply_text(resumen_prenda(nombre, costo, precio, stock, tienda, f
    else:
        await msg.reply_text("Error al guardar en Notion. Intenta de nuevo.")

# ============================================================
# CONVERSATION HANDLER — NUEVA PRENDA GUIADO (v4.2)
# ============================================================
async def cmd_nueva_prenda(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data.clear()
    await _reply(update, "□ *Nueva prenda*\n\nEscribe el *nombre* de la prenda:")
    return NP_NOMBRE

async def np_recibir_nombre(update: Update, context: ContextTypes.DEFAULT_TYPE):
    nombre = update.message.text.strip()
    if len(nombre) < 3:
        await update.message.reply_text("El nombre es muy corto. Escribe al menos 3
        return NP_NOMBRE
    context.user_data["np_nombre"] = nombre
    await update.message.reply_text(f"Prenda: *{nombre}*\n\n□ Escribe el *costo tota
    return NP_COSTO

async def np_recibir_costo(update: Update, context: ContextTypes.DEFAULT_TYPE):
    texto = update.message.text.strip().replace(",",".")
    try:
        costo = float(texto)
        assert costo > 0
    except:
        await update.message.reply_text("Escribe solo el número del costo. Ejemplo:
        return NP_COSTO
    context.user_data["np_costo_total"] = costo
    # precio sugerido (se calcula más tarde cuando se tenga el stock)
    context.user_data["np_costo_guardado"] = costo
    await update.message.reply_text(
        f"Costo total: S/{costo:.0f}\n\n"
        "□ ¿Cuál es el *precio de venta* por unidad?\n"
        "Escribe el precio o presiona el botón para calcular automáticamente:",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("□ Precio sugerido por el bot", callback_data="np_
            [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
```

```
        ])
    )
    return NP_PRECIO

async def np_recibir_precio_auto(update: Update, context: ContextTypes.DEFAULT_TYPE)
    """El precio se calculará tras conocer el stock; guardamos bandera."""
    query = update.callback_query
    await query.answer()
    context.user_data["np_precio_auto"] = True
    await query.edit_message_text(
        "✅ Precio: lo calcularé automáticamente al finalizar.\n\n"
        "□ ¿En qué *unidad* compraste el stock?",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("Docenas □", callback_data="np_stock_doc")],
            [InlineKeyboardButton("Unidades □", callback_data="np_stock_uni")],
            [InlineKeyboardButton("⬅ Volver", callback_data="np_volver_precio")],
            [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
        ])
    )
    return NP_STOCK_TIPO

async def np_recibir_precio_manual(update: Update, context: ContextTypes.DEFAULT_TYP
    texto = update.message.text.strip().replace(",",".")
    try:
        precio = float(texto)
        assert precio > 0
    except:
        await update.message.reply_text("Escribe solo el número. Ejemplo: 130")
        return NP_PRECIO
    context.user_data["np_precio"] = precio
    context.user_data["np_precio_auto"] = False
    await update.message.reply_text(
        f"Precio: S/{precio:.0f}\n\n"
        "□ ¿En qué *unidad* compraste el stock?",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("Docenas □", callback_data="np_stock_doc")],
            [InlineKeyboardButton("Unidades □", callback_data="np_stock_uni")],
            [InlineKeyboardButton("⬅ Volver", callback_data="np_volver_precio")],
            [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
        ])
    )
    return NP_STOCK_TIPO

async def np_elegir_stock_tipo(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "np_stock_doc":
        teclado = InlineKeyboardMarkup([
            [InlineKeyboardButton("1 doc (12)", callback_data="np_doc_1"),
             InlineKeyboardButton("2 doc (24)", callback_data="np_doc_2")],
            [InlineKeyboardButton("3 doc (36)", callback_data="np_doc_3"),
             InlineKeyboardButton("4 doc (48)", callback_data="np_doc_4")],
            [InlineKeyboardButton("5 doc (60)", callback_data="np_doc_5"),
             InlineKeyboardButton("6 doc (72)", callback_data="np_doc_6")],
            [InlineKeyboardButton("⬅ Volver", callback_data="np_volver_stock")],
            [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
```

```
        ])
        await query.edit_message_text("¿Cuántas *docenas* compraste?", reply_markup=
        return NP_STOCK_DOCENAS
    else:
        await query.edit_message_text("Escribe la cantidad de *unidades*:")
        return NP_STOCK_UNICA

async def np_stock_docenas_btn(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    docenas = int(query.data.replace("np_doc_",""))
    unidades = docenas * 12
    context.user_data["np_stock"] = unidades
    await query.edit_message_text(f"Stock: {docenas} doc → {unidades} unidades ✅")
    await _avanzar_a_tienda(update, context)
    return NP_TIENDA

async def np_stock_unidades_txt(update: Update, context: ContextTypes.DEFAULT_TYPE):
    texto = update.message.text.strip()
    try:
        uds = int(texto)
        assert uds > 0
    except:
        await update.message.reply_text("Escribe solo el número entero. Ejemplo: 15
        return NP_STOCK_UNICA
    context.user_data["np_stock"] = uds
    await _avanzar_a_tienda(update, context)
    return NP_TIENDA

async def _avanzar_a_tienda(update, context):
    tiendas = fetch_tiendas_registradas()
    botones = [[InlineKeyboardButton(t, callback_data=f"np_tienda:{t}")] for t in tie
    botones.append([InlineKeyboardButton("✏ Escribir tienda manualmente", callback_
    markup = InlineKeyboardMarkup(botones)
    texto = "□ ¿En qué *tienda* compraste la prenda?"
    if update.callback_query:
        await update.callback_query.message.reply_text(texto, reply_markup=markup)
    else:
        await update.message.reply_text(texto, reply_markup=markup)

async def np_recibir_tienda_btn(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "np_tienda_manual":
        await query.edit_message_text("Escribe el nombre de la tienda:")
        return NP_TIENDA
    tienda = query.data.replace("np_tienda:","")
    context.user_data["np_tienda"] = tienda
    await query.edit_message_text(f"Tienda: {tienda} ✅")
    await _avanzar_a_fecha(update, context)
    return NP_FECHA_MES

async def np_recibir_tienda_txt(update: Update, context: ContextTypes.DEFAULT_TYPE):
    tienda = update.message.text.strip()
    context.user_data["np_tienda"] = tienda
    await _avanzar_a_fecha(update, context)
```

```
    return NP_FECHA_MES

async def _avanzar_a_fecha(update, context):
    from datetime import datetime, timedelta, timezone
    hoy = datetime.now(timezone.utc) - timedelta(hours=5)  # Lima UTC-5
    mes_actual = hoy.strftime("%B %Y")          # ej. "Mayo 2026"
    mes_ant    = (hoy.replace(day=1) - timedelta(days=1)).strftime("%B %Y")
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton(f"□ {mes_ant} (mes anterior)", callback_data="np_mes_a
        [InlineKeyboardButton(f"□ {mes_actual} (mes actual)", callback_data="np_mes_a
        [InlineKeyboardButton("⬅ Volver", callback_data="np_volver_stock")],
        [InlineKeyboardButton("✕ Cancelar", callback_data="menu_inicio")],
    ])
    texto = "□ ¿En qué *mes* compraste la prenda?"
    if update.callback_query:
        await update.callback_query.message.reply_text(texto, reply_markup=teclado)
    else:
        await update.message.reply_text(texto, reply_markup=teclado)

async def np_elegir_mes(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "np_mes_ant":
        await query.edit_message_text(
            "Escribe la fecha de compra (mes anterior):\nFormato: DD/MM/YYYY"
        )
        return NP_FECHA_DIA
    else:
        from datetime import datetime, timedelta, timezone
        hoy    = datetime.now(timezone.utc) - timedelta(hours=5)  # Lima UTC-5
        ayer   = (hoy - timedelta(days=1)).strftime("%d/%m/%Y")
        hoyfmt = hoy.strftime("%d/%m/%Y")
        teclado = InlineKeyboardMarkup([
            [InlineKeyboardButton(f"Ayer ({ayer})",   callback_data=f"np_dia:{ayer}")
            [InlineKeyboardButton(f"Hoy ({hoyfmt})",  callback_data=f"np_dia:{hoyfmt}
            [InlineKeyboardButton("□ Otra fecha",    callback_data="np_dia_manual")],
            [InlineKeyboardButton("⬅ Volver",        callback_data="np_volver_mes")]
            [InlineKeyboardButton("✕ Cancelar",      callback_data="menu_inicio")],
        ])
        await query.edit_message_text("¿Qué día fue la compra?", reply_markup=teclad
        return NP_FECHA_DIA

async def np_recibir_fecha(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    if query:
        await query.answer()
        if query.data == "np_dia_manual":
            await query.edit_message_text("Escribe la fecha:\nFormato: DD/MM/YYYY")
            return NP_FECHA_DIA
        fecha_str = query.data.replace("np_dia:","")
        await query.edit_message_text(f"Fecha: {fecha_str} ✅")
    else:
        fecha_str = update.message.text.strip()
    # Parsear fecha
    from datetime import datetime
    for fmt in ("%d/%m/%Y", "%d-%m-%Y", "%Y-%m-%d"):
```

```
        try:
            dt = datetime.strptime(fecha_str, fmt)
            fecha_iso = dt.strftime("%Y-%m-%d")
            context.user_data["np_fecha"] = fecha_iso
            await _avanzar_a_foto(update, context)
            return NP_FOTO
        except:
            pass
    texto = "Formato incorrecto. Escribe la fecha así: DD/MM/YYYY\nEjemplo: 01/05/202
    if update.callback_query:
        await update.callback_query.message.reply_text(texto)
    else:
        await update.message.reply_text(texto)
    return NP_FECHA_DIA

async def np_volver_precio(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Volver desde Stock tipo → re-pedir precio."""
    query = update.callback_query
    await query.answer()
    context.user_data.pop("np_precio", None)
    context.user_data.pop("np_precio_auto", None)
    costo = context.user_data.get("np_costo_guardado", 0)
    await query.message.reply_text(
        f"Costo total: S/{costo:.0f}\n\n"
        "□ ¿Cuál es el *precio de venta* por unidad?\n"
        "Escribe el precio o presiona el botón para calcular automáticamente:",
        parse_mode="Markdown",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("□ Precio sugerido por el bot", callback_data="np_
            [InlineKeyboardButton("✕ Cancelar", callback_data="menu_inicio")],
        ])
    )
    return NP_PRECIO

async def np_volver_stock(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Volver desde Docenas/Mes → re-pedir tipo de stock."""
    query = update.callback_query
    await query.answer()
    context.user_data.pop("np_stock_tipo", None)
    context.user_data.pop("np_stock", None)
    await query.message.reply_text(
        "□ ¿En qué *unidad* compraste el stock?",
        parse_mode="Markdown",
        reply_markup=InlineKeyboardMarkup([
            [InlineKeyboardButton("Docenas □", callback_data="np_stock_doc")],
            [InlineKeyboardButton("Unidades □", callback_data="np_stock_uni")],
            [InlineKeyboardButton("⬅ Volver", callback_data="np_volver_precio")],
            [InlineKeyboardButton("✕ Cancelar", callback_data="menu_inicio")],
        ])
    )
    return NP_STOCK_TIPO

async def np_volver_mes(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Volver desde Día → re-pedir mes."""
    query = update.callback_query
    await query.answer()
```

```
    context.user_data.pop("np_fecha_mes", None)
    from datetime import datetime, timedelta, timezone
    hoy       = datetime.now(timezone.utc) - timedelta(hours=5)  # Lima UTC-5
    mes_actual = hoy.strftime("%B %Y")
    mes_ant    = (hoy.replace(day=1) - timedelta(days=1)).strftime("%B %Y")
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton(f"□ {mes_ant} (mes anterior)", callback_data="np_mes_a
        [InlineKeyboardButton(f"□ {mes_actual} (mes actual)", callback_data="np_mes_a
        [InlineKeyboardButton("⬅ Volver", callback_data="np_volver_stock")],
        [InlineKeyboardButton("✕ Cancelar", callback_data="menu_inicio")],
    ])
    await query.message.reply_text("□ ¿En qué mes compraste esta prenda?", reply_ma
    return NP_FECHA_MES

async def np_volver_dia(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Volver desde Foto → re-pedir día."""
    query = update.callback_query
    await query.answer()
    context.user_data.pop("np_fecha_dia", None)
    from datetime import datetime, timedelta, timezone
    hoy    = datetime.now(timezone.utc) - timedelta(hours=5)  # Lima UTC-5
    ayer   = (hoy - timedelta(days=1)).strftime("%d/%m/%Y")
    hoyfmt = hoy.strftime("%d/%m/%Y")
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton(f"Ayer ({ayer})",  callback_data=f"np_dia:{ayer}")],
        [InlineKeyboardButton(f"Hoy ({hoyfmt})", callback_data=f"np_dia:{hoyfmt}")],
        [InlineKeyboardButton("□ Otra fecha",   callback_data="np_dia_manual")],
        [InlineKeyboardButton("⬅ Volver",       callback_data="np_volver_mes")],
        [InlineKeyboardButton("✕ Cancelar",      callback_data="menu_inicio")],
    ])
    await query.message.reply_text("□ ¿Qué día compraste esta prenda?", reply_marku
    return NP_FECHA_DIA


async def _avanzar_a_foto(update, context):
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton("□ Sí, quiero agregar foto", callback_data="np_foto_si
        [InlineKeyboardButton("□ No, guardar sin foto",    callback_data="np_foto_no"
        [InlineKeyboardButton("⬅ Volver", callback_data="np_volver_dia")],
        [InlineKeyboardButton("✕ Cancelar", callback_data="menu_inicio")],
    ])
    texto = "¿Quieres agregar una *foto* de la prenda? (opcional)"
    if update.callback_query:
        await update.callback_query.message.reply_text(texto, reply_markup=teclado)
    else:
        await update.message.reply_text(texto, reply_markup=teclado)

async def np_elegir_foto(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "np_foto_si":
        await query.edit_message_text("□ Envía la foto de la prenda:")
        return NP_FOTO
    else:
        await _guardar_nueva_prenda(update, context, foto_url=None)
        return ConversationHandler.END
```

```
async def np_recibir_foto(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg = update.message
    if not msg.photo:
        await msg.reply_text("Envía una foto. Si no tienes, escribe /cancelar")
        return NP_FOTO
    await msg.reply_text("Subiendo foto...")
    foto    = msg.photo[-1]
    archivo = await context.bot.get_file(foto.file_id)
    img_bytes = await archivo.download_as_bytearray()
    foto_url  = subir_imagen(bytes(img_bytes))
    await _guardar_nueva_prenda(update, context, foto_url=foto_url)
    return ConversationHandler.END

async def _guardar_nueva_prenda(update, context, foto_url=None):
    from datetime import datetime
    nombre     = context.user_data.get("np_nombre","")
    costo      = context.user_data.get("np_costo_total", 0)
    stock      = context.user_data.get("np_stock", 0)
    tienda     = context.user_data.get("np_tienda","")
    fecha      = context.user_data.get("np_fecha","")
    precio_auto= context.user_data.get("np_precio_auto", False)
    if precio_auto:
        costo_u = costo / stock if stock > 0 else costo
        precio  = calcular_precio_sugerido(costo_u)
    else:
        precio  = context.user_data.get("np_precio", 0)
    aviso_precio = f"\n□ Precio sugerido automático: S/{precio:.0f}" if precio_auto
    await (update.callback_query.message if update.callback_query else update.messag
        "Guardando en inventario..."
    )
    exito = crear_prenda_notion(nombre, costo, precio, stock, foto_url, tienda, fec
    teclado_post = InlineKeyboardMarkup([
        [InlineKeyboardButton("➕ Agregar otra prenda", callback_data="menu_nueva_gui
        [InlineKeyboardButton("□ Volver al menú",      callback_data="menu_inicio")]
    ])
    if exito:
        msg_ok = resumen_prenda(nombre, costo, precio, stock, tienda, fecha) + avis
        await (update.callback_query.message if update.callback_query else update.me
            msg_ok, reply_markup=teclado_post
        )
    else:
        await (update.callback_query.message if update.callback_query else update.me
            "❌ Error al guardar en Notion. Intenta de nuevo.", reply_markup=teclado_
        )
    context.user_data.clear()

# ============================================================
# CONVERSATION HANDLER — SIN FOTO
# ============================================================
async def cmd_sin_foto(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Registro sin foto\n\nEscribe el nombre de la prenda:\n(Car
    return SINFOTO_NOMBRE

async def sinfoto_recibir_nombre(update: Update, context: ContextTypes.DEFAULT_TYPE
    nombre = update.message.text.strip()
```

```
    if len(nombre) < 3:
        await update.message.reply_text(
            "El nombre es muy corto. Escribe un nombre mas descriptivo.\nEjemplo: C
        )
        return SINFOTO_NOMBRE
    context.user_data["nombre_prenda"] = nombre
    await update.message.reply_text(
        f"Prenda: {nombre}\n\nAhora escribe:\n  costo, precio, stock\n\nOpcionales:\
        "  , tienda, DD/MM/YYYY\n\nEjemplo: 96, 130, 12, Gamarra, 01/05/2026"
    )
    return SINFOTO_DATOS

async def sinfoto_recibir_datos(update: Update, context: ContextTypes.DEFAULT_TYPE):
    nombre = context.user_data.get("nombre_prenda", "Sin nombre")
    datos  = parsear_caption(f"{nombre}, {update.message.text.strip()}")
    if not datos:
        await update.message.reply_text(
            "No entendi los datos. Escribelos asi:\n  costo, precio, stock\n\n"
            "(Escribe - como precio si aun no lo tienes definido)"
        )
        return SINFOTO_DATOS
    nombre, costo, precio, stock, tienda, fecha = datos
    if precio is None:
        costo_unit = costo / stock if stock > 0 else costo
        precio = calcular_precio_sugerido(costo_unit)
        precio_auto = True
    else:
        precio_auto = False
    if costo <= 0 or precio <= 0 or stock <= 0:
        await update.message.reply_text("El costo, precio y stock deben ser mayores
        return SINFOTO_DATOS
    await update.message.reply_text("Guardando en inventario...")
    exito = crear_prenda_notion(nombre, costo, precio, stock, None, tienda, fecha)
    if exito:
        await update.message.reply_text(
            resumen_prenda(nombre, costo, precio, stock, tienda, fecha) +
            (f"\n\n⚠ Precio calculado automáticamente: S/{precio:.0f}" if precio_aut
            "\n\nSin foto por ahora. Puedes adjuntarla con /adjfoto"
        )
    else:
        await update.message.reply_text("Error al guardar en Notion.")
    context.user_data.clear()
    return ConversationHandler.END

# ============================================================
# CONVERSATION HANDLER — ADJUNTAR FOTO
# ============================================================
async def cmd_adj_foto(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Escribe el nombre (o parte) de la prenda:\n(Cancela con /c
    return ADJFOTO_BUSCAR

async def adjfoto_buscar_prenda(update: Update, context: ContextTypes.DEFAULT_TYPE):
    termino = update.message.text.strip()
    if len(termino) < 2:
        await update.message.reply_text("Escribe al menos 2 letras para buscar.")
        return ADJFOTO_BUSCAR
```

```
    await update.message.reply_text(f"Buscando '{termino}'...")
    prendas = buscar_prendas_notion(termino)
    if not prendas:
        await update.message.reply_text(f"No encontre prendas con '{termino}'. Inter
        return ADJFOTO_BUSCAR
    if len(prendas) > 10:
        await update.message.reply_text(f"Mas de 10 resultados para '{termino}'. Se
        return ADJFOTO_BUSCAR
    if len(prendas) == 1:
        context.user_data["prenda_adj"] = prendas[0]
        await update.message.reply_text(f"Encontre: {prendas[0]['nombre']}\n\nAhora
        return ADJFOTO_RECIBIR
    context.user_data["prendas_encontradas"] = {p["id"]: p for p in prendas}
    await update.message.reply_text(
        f"Encontre {len(prendas)} prendas. ¿A cual adjuntas la foto?",
        reply_markup=teclado_lista_prendas(prendas, "sel_adjfoto")
    )
    return ADJFOTO_CONFIRMAR

async def adjfoto_confirmar_prenda(update: Update, context: ContextTypes.DEFAULT_TYP
    query = update.callback_query
    await query.answer()
    if query.data == "cancelar":
        await query.edit_message_text("Operacion cancelada.")
        context.user_data.clear()
        return ConversationHandler.END
    page_id = query.data.replace("sel_adjfoto:", "")
    prenda  = context.user_data.get("prendas_encontradas", {}).get(page_id)
    if not prenda:
        await query.edit_message_text("Error. Intenta de nuevo con /adjfoto")
        return ConversationHandler.END
    context.user_data["prenda_adj"] = prenda
    await query.edit_message_text(f"{prenda['nombre']} seleccionada.\n\nAhora enviam
    return ADJFOTO_RECIBIR

async def adjfoto_recibir_foto(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg    = update.message
    prenda = context.user_data.get("prenda_adj")
    if not prenda:
        await msg.reply_text("Algo salio mal. Empieza de nuevo con /adjfoto")
        return ConversationHandler.END
    if not msg.photo:
        await msg.reply_text("Eso no es una foto. Enviame una imagen.")
        return ADJFOTO_RECIBIR
    await msg.reply_text("Subiendo foto...")
    foto      = msg.photo[-1]
    archivo   = await context.bot.get_file(foto.file_id)
    img_bytes = await archivo.download_as_bytearray()
    foto_url  = subir_imagen(bytes(img_bytes))
    if not foto_url:
        await msg.reply_text("Error al subir la foto. Intenta de nuevo.")
        return ADJFOTO_RECIBIR
    exito = actualizar_prenda_notion(prenda["id"], {"Foto": {"url": foto_url}})
    if exito:
        await msg.reply_text(f"✅ Foto adjuntada correctamente a {prenda['nombre']}")
    else:
```

```
        await msg.reply_text("Error al actualizar en Notion.")
    context.user_data.clear()
    return ConversationHandler.END

# ============================================================
# CONVERSATION HANDLER — REGISTRAR VENTA
# ============================================================
async def cmd_vendi(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Registrar venta\n\nEscribe el nombre (o parte) de la prenc
    return VENTA_BUSCAR

async def venta_buscar_prenda(update: Update, context: ContextTypes.DEFAULT_TYPE):
    termino = update.message.text.strip()
    if len(termino) < 2:
        await update.message.reply_text("Escribe al menos 2 letras para buscar.")
        return VENTA_BUSCAR
    await update.message.reply_text(f"Buscando '{termino}'...")
    prendas    = buscar_prendas_notion(termino)
    disponibles = [p for p in prendas if p["stock"] > 0]
    agotadas    = [p for p in prendas if p["stock"] == 0]
    if not prendas:
        await update.message.reply_text(f"No encontre prendas con '{termino}'.")
        return VENTA_BUSCAR
    if not disponibles:
        nombres = "\n".join(f"- {p['nombre']}" for p in agotadas)
        await update.message.reply_text(f"Todas las prendas encontradas están agotada
        return VENTA_BUSCAR
    if len(disponibles) > 10:
        await update.message.reply_text("Mas de 10 resultados. Se más específica.")
        return VENTA_BUSCAR
    if len(disponibles) == 1:
        context.user_data["prenda_venta"] = disponibles[0]
        p = disponibles[0]
        await update.message.reply_text(
            f"{p['nombre']}\n{p['stock']} unidades disponibles | Precio: S/{p['preci
        )
        return VENTA_CANTIDAD
    context.user_data["prendas_encontradas"] = {p["id"]: p for p in disponibles}
    await update.message.reply_text(
        f"{len(disponibles)} prendas encontradas. ¿Cuál vendiste?",
        reply_markup=teclado_lista_prendas(disponibles, "sel_venta")
    )
    return VENTA_CONFIRMAR

async def venta_confirmar_prenda(update: Update, context: ContextTypes.DEFAULT_TYPE)
    query = update.callback_query
    await query.answer()
    if query.data == "cancelar":
        await query.edit_message_text("Operacion cancelada.")
        context.user_data.clear()
        return ConversationHandler.END
    page_id = query.data.replace("sel_venta:", "")
    prenda  = context.user_data.get("prendas_encontradas", {}).get(page_id)
    if not prenda:
        await query.edit_message_text("Error. Intenta de nuevo con /venta")
        return ConversationHandler.END
```

```
    context.user_data["prenda_venta"] = prenda
    await query.edit_message_text(
        f"{prenda['nombre']}\n{prenda['stock']} unidades disponibles | Precio: S/{pr
    )
    return VENTA_CANTIDAD

async def venta_recibir_cantidad(update: Update, context: ContextTypes.DEFAULT_TYPE)
    prenda = context.user_data.get("prenda_venta")
    if not prenda:
        await update.message.reply_text("Algo salio mal. Empieza de nuevo con /venta
        return ConversationHandler.END
    try:
        cantidad = int(update.message.text.strip())
        if cantidad <= 0:
            raise ValueError
    except ValueError:
        await update.message.reply_text("Escribe un numero entero mayor a 0.")
        return VENTA_CANTIDAD
    if cantidad > prenda["stock"]:
        await update.message.reply_text(f"Solo hay {prenda['stock']} unidades. ¿Cuán
        return VENTA_CANTIDAD
    context.user_data["venta_cantidad"] = cantidad
    precio_std = prenda["precio"]
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton(f"Precio normal S/{precio_std:.0f}", callback_data="pr
        [InlineKeyboardButton("Otro precio", callback_data="precio_venta_otro")],
        [InlineKeyboardButton("⬅ Volver", callback_data="volver_cantidad")],
        [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
    ])
    await update.message.reply_text(f"¿A qué precio vendiste cada una?", reply_mark
    return VENTA_PRECIO

async def venta_recibir_precio(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query  = update.callback_query
    await query.answer()
    prenda = context.user_data.get("prenda_venta")
    if query.data == "precio_venta_std":
        context.user_data["venta_precio_real"] = prenda["precio"]
        await pregunta_descuento(query.message)
        return VENTA_DESCUENTO
    else:
        await query.message.reply_text(
            f"¿A cuánto vendiste cada prenda? (precio normal: S/{prenda['precio']:.0
        )
        return VENTA_PRECIO

async def venta_recibir_precio_manual(update: Update, context: ContextTypes.DEFAULT_
    try:
        precio_real = float(update.message.text.strip().replace(",", "."))
        if precio_real <= 0:
            raise ValueError
    except ValueError:
        await update.message.reply_text("Escribe un precio válido. Ejemplo: 35 o 35.
        return VENTA_PRECIO
    context.user_data["venta_precio_real"] = precio_real
    await pregunta_descuento(update.message)
```

```
    return VENTA_DESCUENTO

async def pregunta_fecha(msg_obj):
    from datetime import datetime, timedelta, timezone
    ahora    = datetime.now(timezone.utc) - timedelta(hours=5)  # Lima UTC-5
    hoy      = ahora.strftime("%d/%m/%Y")
    ayer     = (ahora - timedelta(days=1)).strftime("%d/%m/%Y")
    dia_hoy  = ahora.strftime("%a %d/%m").capitalize()
    dia_ayer = (ahora - timedelta(days=1)).strftime("%a %d/%m").capitalize()
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton(f"Hoy ({dia_hoy})", callback_data="fecha_venta_hoy")],
        [InlineKeyboardButton(f"Ayer ({dia_ayer})", callback_data="fecha_venta_ayer")
        [InlineKeyboardButton("Otra fecha", callback_data="fecha_venta_otra")],
        [InlineKeyboardButton("⬅ Volver", callback_data="volver_descuento")],
        [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
    ])
    await msg_obj.reply_text("¿Cuándo fue la venta?", reply_markup=teclado)

async def venta_recibir_fecha(update: Update, context: ContextTypes.DEFAULT_TYPE):
    from datetime import datetime, timedelta, timezone
    query = update.callback_query
    await query.answer()
    ahora = datetime.now(timezone.utc) - timedelta(hours=5)  # Lima UTC-5
    if query.data == "fecha_venta_hoy":
        context.user_data["venta_fecha"] = ahora.strftime("%Y-%m-%d")
        await venta_pedir_cliente(query.message, context)
        return VENTA_CLIENTE
    elif query.data == "fecha_venta_ayer":
        context.user_data["venta_fecha"] = (ahora - timedelta(days=1)).strftime("%Y-%
        await venta_pedir_cliente(query.message, context)
        return VENTA_CLIENTE
    else:
        await query.message.reply_text("Escribe la fecha en formato DD/MM/YYYY:")
        return VENTA_FECHA

async def venta_recibir_fecha_manual(update: Update, context: ContextTypes.DEFAULT_T
    try:
        fecha = datetime.strptime(update.message.text.strip(), "%d/%m/%Y").strftime
        context.user_data["venta_fecha"] = fecha
        await venta_pedir_cliente(update.message, context)
        return VENTA_CLIENTE
    except ValueError:
        await update.message.reply_text("Formato incorrecto. Usa DD/MM/YYYY. Ejemplo
        return VENTA_FECHA


def obtener_clientes_previos() -> list:
    """Obtiene lista de clientes únicos registrados en la BD de ventas."""
    if not NOTION_VENTAS_ID:
        return []
    url = f"https://api.notion.com/v1/databases/{NOTION_VENTAS_ID}/query"
    payload = {"page_size": 100, "sorts": [{"property": "Fecha", "direction": "desc
    clientes = set()
    try:
        r = requests.post(url, headers=NOTION_HEADERS, json=payload)
        if r.status_code != 200:
```

```
            return []
        for page in r.json().get("results", []):
            props = page.get("properties", {})
            cliente_r = props.get("Cliente", {}).get("rich_text", [])
            if cliente_r:
                nombre = cliente_r[0]["text"]["content"].strip()
                if nombre:
                    clientes.add(nombre)
    except Exception:
        return []
    return sorted(clientes)

async def venta_pedir_cliente(msg_obj, context):
    clientes_prev = obtener_clientes_previos()
    filas = []
    # Todos los clientes, 2 por fila — Telegram soporta hasta ~100 botones
    for i in range(0, len(clientes_prev), 2):
        fila = [InlineKeyboardButton(clientes_prev[i], callback_data=f"cliente_prev_{
        if i + 1 < len(clientes_prev):
            fila.append(InlineKeyboardButton(clientes_prev[i+1], callback_data=f"clie
        filas.append(fila)
    # Botones fijos al final
    filas.append([InlineKeyboardButton("✏ Nueva clienta", callback_data="cliente_nue
    filas.append([InlineKeyboardButton("Sin nombre", callback_data="cliente_sin_nomb
    filas.append([InlineKeyboardButton("⬅ Volver", callback_data="volver_fecha")])
    filas.append([InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")])
    texto = "□ *¿A quién le vendiste?*"
    if clientes_prev:
        texto += f"\n_{len(clientes_prev)} clientas registradas. Elige o agrega una r
    teclado = InlineKeyboardMarkup(filas)
    await msg_obj.reply_text(texto, reply_markup=teclado, parse_mode="Markdown")

async def venta_recibir_cliente(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    if query:
        await query.answer()
        data = query.data
        if data == "cliente_sin_nombre":
            context.user_data["venta_cliente"] = ""
            await _finalizar_venta(update, context)
            return ConversationHandler.END
        elif data == "cliente_nueva":
            await query.message.reply_text("✏ Escribe el nombre de la nueva clienta:
            return VENTA_CLIENTE
        elif data.startswith("cliente_prev:"):
            key = data.split(":",1)[1]
            context.user_data["venta_cliente"] = context.user_data.get("clientes_prev
            await _finalizar_venta(update, context)
            return ConversationHandler.END
    else:
        context.user_data["venta_cliente"] = update.message.text.strip() if update.m
        msg_obj = update.message or update.callback_query.message
        await _finalizar_venta(update, context)
        return ConversationHandler.END

async def venta_recibir_cliente_texto(update: Update, context: ContextTypes.DEFAULT_
```

```
    context.user_data["venta_cliente"] = update.message.text.strip()
    await _finalizar_venta(update, context)
    return ConversationHandler.END

async def pregunta_descuento(msg_obj):
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton("Sin descuento", callback_data="descuento_no")],
        [InlineKeyboardButton("Sí hubo descuento", callback_data="descuento_si")],
        [InlineKeyboardButton("⬅ Volver", callback_data="volver_precio")],
        [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
    ])
    await msg_obj.reply_text("¿Aplicaste algún descuento?", reply_markup=teclado)

async def venta_recibir_descuento(update: Update, context: ContextTypes.DEFAULT_TYPE
    query = update.callback_query
    await query.answer()
    if query.data == "descuento_no":
        context.user_data["venta_descuento"] = 0.0
        await pregunta_fecha(query.message)
        return VENTA_FECHA
    else:
        precio_actual = context.user_data.get("venta_precio_real", 0)
        await query.message.reply_text(
            f"Precio registrado: S/{precio_actual:.0f}\n"
            "¿Cuánto fue el descuento? (en soles)\n"
            "Ejemplo: 1 o 2"
        )
        return VENTA_DESCUENTO

async def venta_recibir_descuento_monto(update: Update, context: ContextTypes.DEFAUL
    try:
        descuento = float(update.message.text.strip().replace(",", "."))
        if descuento < 0:
            raise ValueError
    except ValueError:
        await update.message.reply_text("Escribe un monto válido. Ejemplo: 5 o 10.50
        return VENTA_DESCUENTO
    context.user_data["venta_descuento"] = descuento
    precio_actual = context.user_data.get("venta_precio_real", 0)
    await update.message.reply_text(
        f"✅ Descuento de S/{descuento:.0f} registrado. "
        f"Precio final: S/{precio_actual - descuento:.0f}"
    )
    await pregunta_fecha(update.message)
    return VENTA_FECHA

async def venta_volver_cantidad(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Volver desde Precio → re-pedir cantidad."""
    query = update.callback_query
    await query.answer()
    context.user_data.pop("venta_precio_real", None)
    prenda = context.user_data.get("prenda_venta")
    await query.message.reply_text(
        f"¿Cuántas unidades de *{prenda['nombre']}* vendiste?\nEscribe solo el númer
        parse_mode="Markdown"
    )
```

```
    return VENTA_CANTIDAD

async def venta_volver_precio(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Volver desde Descuento → re-pedir precio."""
    query = update.callback_query
    await query.answer()
    context.user_data.pop("venta_descuento", None)
    prenda = context.user_data.get("prenda_venta")
    precio_std = prenda["precio"]
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton(f"Precio normal S/{precio_std:.0f}", callback_data="pr
        [InlineKeyboardButton("Otro precio", callback_data="precio_venta_otro")],
        [InlineKeyboardButton("⬅ Volver", callback_data="volver_cantidad")],
        [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
    ])
    await query.message.reply_text("¿A qué precio vendiste cada una?", reply_markup=
    return VENTA_PRECIO

async def venta_volver_descuento(update: Update, context: ContextTypes.DEFAULT_TYPE)
    """Volver desde Fecha → re-pedir descuento."""
    query = update.callback_query
    await query.answer()
    context.user_data.pop("venta_fecha", None)
    await pregunta_descuento(query.message)
    return VENTA_DESCUENTO

async def venta_volver_fecha(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Volver desde Cliente → re-pedir fecha."""
    query = update.callback_query
    await query.answer()
    context.user_data.pop("venta_cliente", None)
    await pregunta_fecha(query.message)
    return VENTA_FECHA


async def _finalizar_venta(update, context):
    prenda      = context.user_data.get("prenda_venta")
    cantidad    = context.user_data.get("venta_cantidad")
    precio_real = context.user_data.get("venta_precio_real")
    fecha       = context.user_data.get("venta_fecha")
    descuento   = context.user_data.get("venta_descuento", 0.0)
    cliente     = context.user_data.get("venta_cliente", "")
    if not all([prenda, cantidad, precio_real, fecha]):
        await _reply(update, "Datos incompletos. Intenta de nuevo con /venta")
        context.user_data.clear()
        return
    costo_u      = prenda["costo_u"]
    ganancia_tot = (precio_real - descuento - costo_u) * cantidad
    nuevo_stock  = prenda["stock"] - cantidad
    actualizar_prenda_notion(prenda["id"], {"Stock": {"number": nuevo_stock}})
    import asyncio, functools
    loop = asyncio.get_event_loop()
    await loop.run_in_executor(None, functools.partial(
        registrar_venta_notion,
        nombre_prenda=prenda["nombre"], cantidad=cantidad,
        precio_real=precio_real, costo_u=costo_u,
```

```
        ganancia=ganancia_tot, cliente=cliente, fecha_venta=fecha,
        descuento=descuento,
    ))
    descuento_linea = f"\nDescuento:  -S/{descuento:.0f}" if descuento > 0 else ""
    msg = (
        f"✅ Venta registrada!\n\n"
        f"Prenda: {prenda['nombre']}\n"
        f"Vendidas: {cantidad} uds a S/{precio_real:.0f}"
        f"{descuento_linea}\n"
        f"Ganancia: S/{ganancia_tot:.0f}\n"
        f"Stock restante: {nuevo_stock} uds"
    )
    if nuevo_stock == 0:
        msg += "\n\n⚠ ¡Prenda AGOTADA! Considera reponerla."
    elif nuevo_stock <= 3:
        msg += f"\n\n⚡ Stock bajo: solo quedan {nuevo_stock} uds."
    teclado_post = InlineKeyboardMarkup([
        [InlineKeyboardButton("□ Registrar otra venta", callback_data="menu_venta")],
        [InlineKeyboardButton("□ Volver al menú",        callback_data="menu_inicio")
    ])
    context.user_data.clear()
    if update.callback_query:
        await update.callback_query.message.reply_text(msg, reply_markup=teclado_pos
    elif update.message:
        await update.message.reply_text(msg, reply_markup=teclado_post)
    else:
        await _reply(update, msg, reply_markup=teclado_post)

# ═══════════════════════════════════════════════════════════
# CONVERSATION HANDLER — CONSULTAR PRENDA
# ═══════════════════════════════════════════════════════════
async def cmd_stock(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Consultar prenda\n\nEscribe el nombre (o parte):")
    return STOCK_BUSCAR

async def stock_buscar_prenda(update: Update, context: ContextTypes.DEFAULT_TYPE):
    termino = update.message.text.strip()
    if len(termino) < 2:
        await update.message.reply_text("Escribe al menos 2 letras.")
        return STOCK_BUSCAR
    await update.message.reply_text(f"Buscando '{termino}'...")
    prendas = buscar_prendas_notion(termino)
    if not prendas:
        await update.message.reply_text(f"No encontre prendas con '{termino}'.")
        return STOCK_BUSCAR
    if len(prendas) == 1:
        await update.message.reply_text(_formato_stock(prendas[0]))
        return ConversationHandler.END
    context.user_data["prendas_encontradas"] = {p["id"]: p for p in prendas}
    await update.message.reply_text(
        f"Encontre {len(prendas)} prendas. ¿Cuál quieres consultar?",
        reply_markup=teclado_lista_prendas(prendas, "sel_stock")
    )
    return STOCK_CONFIRMAR

async def stock_confirmar_prenda(update: Update, context: ContextTypes.DEFAULT_TYPE)
```

```
    query = update.callback_query
    await query.answer()
    if query.data == "cancelar":
        await query.edit_message_text("Cancelado.")
        context.user_data.clear()
        return ConversationHandler.END
    page_id = query.data.replace("sel_stock:", "")
    prenda  = context.user_data.get("prendas_encontradas", {}).get(page_id)
    if not prenda:
        await query.edit_message_text("Error. Intenta de nuevo con /prenda")
        return ConversationHandler.END
    await query.edit_message_text(_formato_stock(prenda))
    context.user_data.clear()
    return ConversationHandler.END

# ============================================================
# CONVERSATION HANDLER — EDITAR PRENDA
# ============================================================
# ============================================================
# CONVERSATION HANDLER — ELIMINAR VENTA
# ============================================================
async def cmd_eliminar_venta(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Eliminar venta\n\nEscribe el nombre (o parte) de la prenda
    return ELIMINAR_VENTA_BUSCAR

async def eliminar_venta_buscar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    termino = update.message.text.strip()
    if len(termino) < 2:
        await update.message.reply_text("Escribe al menos 2 letras.")
        return ELIMINAR_VENTA_BUSCAR
    await update.message.reply_text("Buscando ventas...")
    ventas = buscar_ventas_notion(termino)
    if not ventas:
        await update.message.reply_text("No encontré ventas con ese nombre.")
        return ELIMINAR_VENTA_BUSCAR
    context.user_data["ventas_encontradas"] = {v["id"]: v for v in ventas}
    botones = []
    for v in ventas[:10]:
        botones.append([InlineKeyboardButton(v["label"], callback_data=f"sel_elimven
    botones.append([InlineKeyboardButton("Cancelar", callback_data="cancelar")])
    await update.message.reply_text(
        f"Encontré {len(ventas[:10])} ventas. ¿Cuál quieres eliminar?",
        reply_markup=InlineKeyboardMarkup(botones)
    )
    return ELIMINAR_VENTA_CONFIRMAR

async def eliminar_venta_confirmar(update: Update, context: ContextTypes.DEFAULT_TYP
    query = update.callback_query
    await query.answer()
    if query.data == "cancelar":
        await query.edit_message_text("Operación cancelada.")
        context.user_data.clear()
        return ConversationHandler.END
    page_id = query.data.replace("sel_elimventa:", "")
    venta   = context.user_data.get("ventas_encontradas", {}).get(page_id)
    if not venta:
```

```
        await query.edit_message_text("Error. Intenta de nuevo con /eliminar_venta")
        return ConversationHandler.END
    exito = eliminar_venta_notion(page_id)
    if exito:
        await query.edit_message_text(
            f"✅ Venta eliminada y stock restaurado.\n\n"
            f"Prenda: {venta['nombre']}\n"
            f"Cantidad: {venta['cantidad']} uds\n"
            f"Stock restaurado: +{venta['cantidad']} uds"
        )
    else:
        await query.edit_message_text("❌ Error al eliminar la venta. Intenta de nuev
    context.user_data.clear()
    return ConversationHandler.END

async def cmd_editar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "✏ Actualizar / Eliminar prenda\n\nEscribe el nombre (o par
    return EDITAR_BUSCAR

async def editar_buscar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    termino = update.message.text.strip()
    if len(termino) < 2:
        await update.message.reply_text("Escribe al menos 2 letras.")
        return EDITAR_BUSCAR
    prendas = buscar_prendas_notion(termino)
    if not prendas:
        await update.message.reply_text(f"No encontre prendas con '{termino}'.")
        return EDITAR_BUSCAR
    if len(prendas) == 1:
        context.user_data["prenda_editar"] = prendas[0]
        teclado = InlineKeyboardMarkup([
            [InlineKeyboardButton("□ Nombre",  callback_data="ec_nombre"),
             InlineKeyboardButton("□ Costo",   callback_data="ec_costo")],
            [InlineKeyboardButton("□ Precio",  callback_data="ec_precio"),
             InlineKeyboardButton("□ Stock",   callback_data="ec_stock")],
            [InlineKeyboardButton("□ Tienda",  callback_data="ec_tienda"),
             InlineKeyboardButton("□ Fecha",   callback_data="ec_fecha")],
            [InlineKeyboardButton("□ Eliminar esta prenda", callback_data="ec_elimin
            [InlineKeyboardButton("❌ Cancelar",             callback_data="cancelar"
        ])
        await update.message.reply_text(
            f"Prenda: *{prendas[0]['nombre']}*\n\n¿Qué quieres hacer?",
            reply_markup=teclado,
            parse_mode="Markdown"
        )
        return EDITAR_CAMPO
    context.user_data["prendas_encontradas"] = {p["id"]: p for p in prendas}
    await update.message.reply_text(
        f"Encontre {len(prendas)} prendas. ¿Cuál quieres editar?",
        reply_markup=teclado_lista_prendas(prendas, "sel_editar")
    )
    return EDITAR_CONFIRMAR

async def editar_confirmar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
```

```
    if query.data == "cancelar":
        await query.edit_message_text("Cancelado.")
        context.user_data.clear()
        return ConversationHandler.END
    page_id = query.data.replace("sel_editar:", "")
    prenda  = context.user_data.get("prendas_encontradas", {}).get(page_id)
    if not prenda:
        await query.edit_message_text("Error.")
        return ConversationHandler.END
    context.user_data["prenda_editar"] = prenda
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton("□ Nombre",  callback_data="ec_nombre"),
         InlineKeyboardButton("□ Costo",   callback_data="ec_costo")],
        [InlineKeyboardButton("□ Precio",  callback_data="ec_precio"),
         InlineKeyboardButton("□ Stock",   callback_data="ec_stock")],
        [InlineKeyboardButton("□ Tienda",  callback_data="ec_tienda"),
         InlineKeyboardButton("□ Fecha",   callback_data="ec_fecha")],
        [InlineKeyboardButton("□ Eliminar esta prenda", callback_data="ec_eliminar")
        [InlineKeyboardButton("✖ Cancelar",             callback_data="cancelar")],
    ])
    await query.edit_message_text(
        f"Prenda: *{prenda['nombre']}*\n\n¿Qué quieres hacer?",
        reply_markup=teclado
    )
    return EDITAR_CAMPO

async def editar_campo(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    prenda = context.user_data.get("prenda_editar")

    if query.data == "cancelar":
        await query.edit_message_text("Cancelado.")
        context.user_data.clear()
        return ConversationHandler.END

    # ── Eliminar prenda ──────────────────────────────────────
    if query.data == "ec_eliminar":
        teclado = InlineKeyboardMarkup([
            [InlineKeyboardButton("✅ Sí, eliminar definitivamente", callback_data="e
            [InlineKeyboardButton("✖ Cancelar",                     callback_data="ca
        ])
        await query.edit_message_text(
            f"⚠ ¿Segura que quieres eliminar *{prenda['nombre']}*?\n"
            "Esto no se puede deshacer.",
            reply_markup=teclado
        )
        return EDITAR_CAMPO

    if query.data == "ec_elim_ok":
        r = requests.patch(
            f"https://api.notion.com/v1/pages/{prenda['id']}",
            headers=NOTION_HEADERS, json={"archived": True}
        )
        if r.status_code == 200:
            await query.edit_message_text(f"□ Prenda *{prenda['nombre']}* eliminada
```

```
        else:
            await query.edit_message_text("✖ Error al eliminar. Intenta de nuevo.")
        context.user_data.clear()
        return ConversationHandler.END

    # ── Editar campo ──────────────────────────────────────
    campo_map = {
        "ec_nombre": "nombre",
        "ec_costo":  "costo",
        "ec_precio": "precio",
        "ec_stock":  "stock",
        "ec_tienda": "tienda",
        "ec_fecha":  "fecha",
    }
    campo = campo_map.get(query.data)
    if not campo:
        await query.edit_message_text("Opción no reconocida.")
        return ConversationHandler.END
    context.user_data["campo_editar"] = campo
    hints = {
        "nombre": "Escribe el nuevo nombre:",
        "costo":  "Escribe el nuevo costo total (S/):",
        "precio": "Escribe el nuevo precio de venta (S/):",
        "stock":  "Escribe el nuevo stock (número entero):",
        "tienda": "Escribe el nombre de la tienda:",
        "fecha":  "Escribe la nueva fecha (DD/MM/YYYY):",
    }
    await query.edit_message_text(hints[campo])
    return EDITAR_VALOR

async def editar_valor(update: Update, context: ContextTypes.DEFAULT_TYPE):
    prenda = context.user_data.get("prenda_editar")
    campo  = context.user_data.get("campo_editar")
    valor  = update.message.text.strip()
    cambios = {}
    try:
        if campo == "nombre":
            cambios["Prenda"] = {"title": [{"text": {"content": valor}}]}
        elif campo == "costo":
            costo = float(valor)
            cambios["Costo"] = {"number": costo}
            if prenda["stock"] > 0:
                cambios["Costo Unitario"] = {"number": round(costo / prenda["stock"]
        elif campo == "precio":
            cambios["Precio"] = {"number": float(valor)}
        elif campo == "stock":
            cambios["Stock"] = {"number": int(valor)}
        elif campo == "tienda":
            cambios["Tienda"] = {"rich_text": [{"text": {"content": valor}}]}
        elif campo == "fecha":
            fecha = datetime.strptime(valor, "%d/%m/%Y").strftime("%Y-%m-%d")
            cambios["Fecha Compra"] = {"date": {"start": fecha}}
    except ValueError:
        await update.message.reply_text("Valor inválido. Intenta de nuevo.")
        return EDITAR_VALOR
    exito = actualizar_prenda_notion(prenda["id"], cambios)
```

```
    if exito:
        await update.message.reply_text(f"✅ '{campo}' actualizado correctamente en {
    else:
        await update.message.reply_text("❌ Error al actualizar. Intenta de nuevo.")
    context.user_data.clear()
    return ConversationHandler.END

# ============================================================
# CONVERSATION HANDLER — ELIMINAR PRENDA
# ============================================================
async def cmd_eliminar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Eliminar prenda\n\nEscribe el nombre (o parte):")
    return ELIMINAR_BUSCAR

async def eliminar_buscar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    termino = update.message.text.strip()
    prendas = buscar_prendas_notion(termino)
    if not prendas:
        await update.message.reply_text(f"No encontre prendas con '{termino}'.")
        return ELIMINAR_BUSCAR
    if len(prendas) == 1:
        context.user_data["prenda_eliminar"] = prendas[0]
        teclado = InlineKeyboardMarkup([
            [InlineKeyboardButton("✅ Sí, eliminar", callback_data="confirmar_elimina
            [InlineKeyboardButton("❌ Cancelar",     callback_data="cancelar")],
        ])
        await update.message.reply_text(
            f"¿Segura que quieres eliminar '{prendas[0]['nombre']}'? Esta acción no
            reply_markup=teclado
        )
        return ELIMINAR_CONFIRMAR
    context.user_data["prendas_encontradas"] = {p["id"]: p for p in prendas}
    await update.message.reply_text(
        f"Encontre {len(prendas)} prendas. ¿Cuál quieres eliminar?",
        reply_markup=teclado_lista_prendas(prendas, "sel_eliminar")
    )
    return ELIMINAR_CONFIRMAR

async def eliminar_confirmar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "cancelar":
        await query.edit_message_text("Cancelado.")
        context.user_data.clear()
        return ConversationHandler.END
    if query.data == "confirmar_eliminar":
        prenda = context.user_data.get("prenda_eliminar")
        r = requests.patch(
            f"https://api.notion.com/v1/pages/{prenda['id']}",
            headers=NOTION_HEADERS,
            json={"archived": True}
        )
        if r.status_code == 200:
            await query.edit_message_text(f"✅ '{prenda['nombre']}' eliminada del inv
        else:
            await query.edit_message_text("❌ Error al eliminar. Intenta de nuevo.")
```

```
        context.user_data.clear()
        return ConversationHandler.END
    page_id = query.data.replace("sel_eliminar:", "")
    prenda  = context.user_data.get("prendas_encontradas", {}).get(page_id)
    if not prenda:
        await query.edit_message_text("Error.")
        return ConversationHandler.END
    context.user_data["prenda_eliminar"] = prenda
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton("✅ Sí, eliminar", callback_data="confirmar_eliminar")
        [InlineKeyboardButton("❌ Cancelar",     callback_data="cancelar")],
    ])
    await query.edit_message_text(
        f"¿Segura que quieres eliminar '{prenda['nombre']}'? Esta acción no se puede
        reply_markup=teclado
    )
    return ELIMINAR_CONFIRMAR

# ============================================================
# CONVERSATION HANDLER — VER FOTO
# ============================================================
async def cmd_ver_foto(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Ver foto\n\nEscribe el nombre (o parte) de la prenda:")
    return FOTO_BUSCAR

async def verfoto_buscar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    termino = update.message.text.strip()
    if len(termino) < 2:
        await update.message.reply_text("Escribe al menos 2 letras.")
        return FOTO_BUSCAR
    prendas = buscar_prendas_notion(termino)
    if not prendas:
        await update.message.reply_text(f"No encontre prendas con '{termino}'.")
        return FOTO_BUSCAR
    if len(prendas) == 1:
        foto_url = obtener_foto_url(prendas[0]["id"])
        if foto_url:
            await update.message.reply_photo(photo=foto_url, caption=prendas[0]["nom
        else:
            await update.message.reply_text(f"'{prendas[0]['nombre']}' no tiene foto
        return ConversationHandler.END
    context.user_data["prendas_encontradas"] = {p["id"]: p for p in prendas}
    await update.message.reply_text(
        f"Encontre {len(prendas)} prendas. ¿De cuál quieres ver la foto?",
        reply_markup=teclado_lista_prendas(prendas, "sel_foto")
    )
    return FOTO_CONFIRMAR

async def verfoto_confirmar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    if query.data == "cancelar":
        await query.edit_message_text("Cancelado.")
        context.user_data.clear()
        return ConversationHandler.END
    page_id = query.data.replace("sel_foto:", "")
```

```
    prenda  = context.user_data.get("prendas_encontradas", {}).get(page_id)
    if not prenda:
        await query.edit_message_text("Error.")
        return ConversationHandler.END
    foto_url = obtener_foto_url(page_id)
    if foto_url:
        await query.message.reply_photo(photo=foto_url, caption=prenda["nombre"])
    else:
        await query.edit_message_text(f"'{prenda['nombre']}' no tiene foto registrad
    context.user_data.clear()
    return ConversationHandler.END

# ============================================================
# CONVERSATION HANDLER — COMPARAR PRENDAS
# ============================================================
async def cmd_comparar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update,
        "Comparar prendas\n\nEscribe los nombres separados por coma.\n"
        "Ejemplo: chompa azul, blusa floral"
    )
    return COMPARAR_BUSCAR

async def comparar_buscar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    terminos = [t.strip() for t in update.message.text.split(",") if t.strip()]
    if len(terminos) < 2:
        await update.message.reply_text("Escribe al menos 2 prendas separadas por co
        return COMPARAR_BUSCAR
    inventario = fetch_inventario_completo() or []
    encontradas = []
    no_encontradas = []
    campos = [
        ("Stock actual", lambda p: f"{p['stock']} uds"),
        ("Precio", lambda p: f"S/{p['precio']:.0f}"),
        ("Costo unit.", lambda p: f"S/{p['costo_u']:.2f}"),
        ("Ganancia/ud", lambda p: f"S/{p['ganancia_u']:.0f}"),
        ("Margen", lambda p: f"{p['margen']}%"),
        ("Vendidas", lambda p: f"{p['vendidas']} uds"),
        ("Ganancia total", lambda p: f"S/{p['ganancia_real']:.0f}"),
    ]
    for termino in terminos:
        res = [p for p in inventario if termino.lower() in p["nombre"].lower()]
        if res:
            encontradas.append(res[0])
        else:
            no_encontradas.append(termino)
    if len(encontradas) < 2:
        await update.message.reply_text(
            f"No encontre suficientes prendas. No encontradas: {', '.join(no_encontr
        )
        return COMPARAR_BUSCAR
    lineas = ["□ Comparación de prendas\n"]
    ids_enc = {p["id"] for p in encontradas}
    detalle = {p["id"]: p for p in encontradas}
    for label, fn in campos:
        valores = "  |  ".join(fn(detalle[p["id"]]) for p in encontradas)
        lineas.append(f"{label:15}: {valores}")
```

```
    nombres = "  vs  ".join(p["nombre"] for p in encontradas)
    lineas.insert(1, nombres + "\n")
    if no_encontradas:
        lineas.append(f"\nNo encontradas: {', '.join(no_encontradas)}")
    await update.message.reply_text("\n".join(lineas))
    return ConversationHandler.END

# ═══════════════════════════════════════════════════════════
# COMANDOS DE REPORTES
# ═══════════════════════════════════════════════════════════
async def cmd_inventario(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Consultando inventario completo...")
    prendas = fetch_inventario_completo()
    if prendas is None:
        await _reply(update, "Error al consultar Notion.")
        return
    if not prendas:
        await _reply(update, "El inventario está vacío.")
        return
    disp     = sum(1 for p in prendas if p["estado"] == "Disponible")
    limitado = sum(1 for p in prendas if p["estado"] == "Stock limitado")
    agotado  = sum(1 for p in prendas if p["estado"] == "Agotado")
    lineas = [
        f"Inventario completo ({len(prendas)} prendas)\n",
        f"Verde (Disponible):      {disp}",
        f"Amarillo (Stock bajo):   {limitado}",
        f"Rojo (Agotado):         {agotado}\n",
    ]
    # Tabla en formato monospace para Telegram
    col_nombre = max(len(p["nombre"]) for p in prendas)
    col_nombre = min(col_nombre, 22)  # máximo 22 chars
    encabezado = f"{'Prenda':<22} {'Stk':>4} {'S/':>5} Est"
    separador  = "─" * (22 + 4 + 5 + 4 + 3)
    filas = [encabezado, separador]
    for p in prendas:
        icono  = "□" if p["estado"] == "Disponible" else ("□" if p["estado"] == "Sto
        nombre = p["nombre"][:22].ljust(22)
        stk    = str(p["stock"]).rjust(4)
        precio = str(int(p["precio"])).rjust(5)
        filas.append(f"{nombre} {stk} {precio} {icono}")
    tabla = "\n".join(filas)
    resumen_hdr = "\n".join(lineas)
    await _reply(update, resumen_hdr + "\n\n<pre>" + tabla + "</pre>", parse_mode="


async def cmd_ganancias_menu(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Submenú financiero con opciones de resumen."""
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton("□ Resumen general",  callback_data="fin_resumen")],
        [InlineKeyboardButton("□ Por margen",        callback_data="fin_pormargen")]
        [InlineKeyboardButton("□ Por tienda",        callback_data="fin_portienda")]
        [InlineKeyboardButton("□ Por fecha",         callback_data="fin_porfecha")],
        [InlineKeyboardButton("✕ Cancelar",          callback_data="menu_inicio")],
    ])
    msg = update.message or update.callback_query.message
    await msg.reply_text("□ *Ganancias — ¿qué quieres ver?*", reply_markup=teclado,
```

```
async def cmd_ganancias_fecha_menu(update: Update, context: ContextTypes.DEFAULT_TYP
    """Submenú de ganancias por fecha."""
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton("□ Hoy",          callback_data="fin_fecha_hoy")],
        [InlineKeyboardButton("□ Ayer",         callback_data="fin_fecha_ayer")],
        [InlineKeyboardButton("□ Esta semana",  callback_data="fin_fecha_semana")],
        [InlineKeyboardButton("□ Este mes",     callback_data="fin_fecha_mes")],
        [InlineKeyboardButton("⬅ Volver",       callback_data="menu_ganancias")],
        [InlineKeyboardButton("✖ Cancelar",     callback_data="menu_inicio")],
    ])
    query = update.callback_query
    await query.edit_message_text("□ *Ganancias por fecha — ¿qué período?*", reply_m

async def cmd_ganancias_por_fecha(update: Update, context: ContextTypes.DEFAULT_TYPE
    """Calcula ganancias netas del período solicitado consultando la BD de ventas."""
    from datetime import date, timedelta
    query = update.callback_query
    await query.answer()
    periodo = query.data  # fin_fecha_hoy / ayer / semana / mes

    hoy = date.today()
    if periodo == "fin_fecha_hoy":
        desde = hasta = hoy
        etiqueta = "Hoy"
    elif periodo == "fin_fecha_ayer":
        desde = hasta = hoy - timedelta(days=1)
        etiqueta = "Ayer"
    elif periodo == "fin_fecha_semana":
        desde = hoy - timedelta(days=hoy.weekday())  # lunes
        hasta = desde + timedelta(days=6)            # domingo
        etiqueta = f"Esta semana ({desde.strftime('%d/%m')} - {hasta.strftime('%d/%m
    elif periodo == "fin_fecha_mes":
        desde = hoy.replace(day=1)
        hasta = hoy
        etiqueta = hoy.strftime("%B %Y").capitalize()
    else:
        return

    # Consultar Notion ventas filtrado por rango de fecha
    if not NOTION_VENTAS_ID:
        await query.message.reply_text("No hay base de datos de ventas configurada."
        return

    url = f"https://api.notion.com/v1/databases/{NOTION_VENTAS_ID}/query"
    payload = {
        "filter": {
            "and": [
                {"property": "Fecha", "date": {"on_or_after": desde.isoformat()}},
                {"property": "Fecha", "date": {"on_or_before": hasta.isoformat()}},
            ]
        },
        "page_size": 100
    }
    await query.message.reply_text(f"Calculando ganancias: {etiqueta}…")
    r = requests.post(url, headers=NOTION_HEADERS, json=payload)
```

```
    if r.status_code != 200:
        await query.message.reply_text("Error al consultar Notion. Intenta de nuevo.
        return

    resultados = r.json().get("results", [])
    if not resultados:
        teclado = InlineKeyboardMarkup([
            [InlineKeyboardButton("⬅ Volver", callback_data="fin_porfecha")],
            [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
        ])
        await query.message.reply_text(
            f"□ *{etiqueta}*\nSin ventas registradas en este período.",
            reply_markup=teclado, parse_mode="Markdown"
        )
        return

    ventas = []
    for page in resultados:
        props = page.get("properties", {})
        try:
            ganancia = props.get("Ganancia", {}).get("number") or 0
            cantidad = props.get("Cantidad", {}).get("number") or 0
            precio   = props.get("Precio real", {}).get("number") or 0
            prenda_r = props.get("Prenda", {}).get("rich_text", [])
            prenda   = prenda_r[0]["text"]["content"] if prenda_r else "Sin nombre"
            cliente_r = props.get("Cliente", {}).get("rich_text", [])
            cliente  = cliente_r[0]["text"]["content"] if cliente_r else ""
            ventas.append({"prenda": prenda, "cantidad": cantidad,
                           "precio": precio, "ganancia": ganancia, "cliente": client
        except Exception:
            continue

    total_gan  = sum(v["ganancia"] for v in ventas)
    total_uds  = sum(v["cantidad"] for v in ventas)
    total_ing  = sum(v["precio"] * v["cantidad"] for v in ventas)
    n_ventas   = len(ventas)

    lineas = [
        f"□ *Ganancias — {etiqueta}*\n",
        f"Ventas registradas: {n_ventas}",
        f"Unidades vendidas:  {total_uds} uds",
        f"Ingresos:           S/{total_ing:.0f}",
        f"Ganancia neta:      S/{total_gan:.0f}\n",
        "Detalle:"
    ]
    for v in ventas:
        cli = f" → {v['cliente']}" if v["cliente"] else ""
        lineas.append(f"  • {v['prenda']} x{v['cantidad']}  +S/{v['ganancia']:.0f}{cli

    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton("⬅ Volver", callback_data="fin_porfecha")],
        [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
    ])
    await query.message.reply_text(
        "\n".join(lineas), reply_markup=teclado, parse_mode="Markdown"
    )
```

```
async def cmd_resumen(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Calculando resumen financiero...")
    prendas = fetch_inventario_completo()
    if prendas is None:
        await _reply(update, "Error al consultar Notion.")
        return
    if not prendas:
        await _reply(update, "El inventario está vacío.")
        return
    total_prendas    = len(prendas)
    disponibles      = sum(1 for p in prendas if p["estado"] == "Disponible")
    limitadas        = sum(1 for p in prendas if p["estado"] == "Stock limitado")
    agotadas         = sum(1 for p in prendas if p["estado"] == "Agotado")
    total_vendidas   = sum(p["vendidas"] for p in prendas)
    ingreso_total    = sum(p["ingreso_real"] for p in prendas)
    ganancia_total   = sum(p["ganancia_real"] for p in prendas)
    inversion_rest   = sum(p["inversion_rest"] for p in prendas)
    valor_stock_rest = sum(p["valor_restante"] for p in prendas)
    ganancia_pot     = sum(p["stock"] * p["ganancia_u"] for p in prendas)
    total_invertido  = sum(p["costo"] for p in prendas)
    roi = round(ganancia_total / total_invertido * 100, 1) if total_invertido > 0 els
    margen_prom = round(sum(p["margen"] for p in prendas) / len(prendas), 1) if pre
    top_ganancia = sorted(prendas, key=lambda p: p["ganancia_real"], reverse=True)[:
    top_margen   = sorted(prendas, key=lambda p: p["margen"], reverse=True)[:3]
    stock_bajo   = [p for p in prendas if 0 < p["stock"] <= 3]
    lineas = [
        "Resumen financiero — Maricuchis Store\n",
        f"Prendas en inventario: {total_prendas}",
        f"  Disponibles:   {disponibles}",
        f"  Stock limitado:{limitadas}",
        f"  Agotadas:      {agotadas}\n",
        "LO QUE YA VENDISTE",
        f"  Unidades vendidas:  {total_vendidas} uds",
        f"  Ingresos generados: S/{ingreso_total:.0f}",
        f"  Ganancia obtenida:  S/{ganancia_total:.0f}",
        f"  ROI sobre inversión:{roi}%\n",
        "LO QUE AÚN TIENES",
        f"  Inversión en stock: S/{inversion_rest:.0f}",
        f"  Valor venta stock:  S/{valor_stock_rest:.0f}",
        f"  Ganancia potencial: S/{ganancia_pot:.0f}\n",
        "EFICIENCIA",
        f"  Total invertido:    S/{total_invertido:.0f}",
        f"  Margen promedio:    {margen_prom}%\n",
        "TOP 3 MÁS RENTABLES (ganancia acumulada)",
    ]
    for i, p in enumerate(top_ganancia, 1):
        lineas.append(f"  {i}. {p['nombre']} — S/{p['ganancia_real']:.0f} ({p['vendic
    lineas += ["", "TOP 3 MEJOR MARGEN"]
    for i, p in enumerate(top_margen, 1):
        lineas.append(f"  {i}. {p['nombre']} — {p['margen']}% (S/{p['ganancia_u']:.0
    if stock_bajo:
        lineas += ["", "⚡ ATENCIÓN — STOCK BAJO (reponer pronto)"]
        for p in stock_bajo:
            lineas.append(f"  {p['nombre']}: solo {p['stock']} uds")
    teclado = InlineKeyboardMarkup([
```

```
        [InlineKeyboardButton("⬅ Volver", callback_data="menu_ganancias")],
        [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
    ])
    msg = update.message or update.callback_query.message
    await msg.reply_text("\n".join(lineas), reply_markup=teclado)

async def cmd_por_margen(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Calculando ranking por margen...")
    prendas = fetch_inventario_completo()
    if not prendas:
        await _reply(update, "Inventario vacío o error.")
        return
    ranking = sorted(prendas, key=lambda p: p["margen"], reverse=True)
    lineas  = [f"Ranking por margen de ganancia ({len(ranking)} prendas)\n"]
    for i, p in enumerate(ranking, 1):
        lineas.append(
            f"{i:2}. {p['nombre']}\n"
            f"    Margen: {p['margen']}% | Ganancia/ud: S/{p['ganancia_u']:.0f} | Sto
        )
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton("⬅ Volver", callback_data="menu_ganancias")],
        [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
    ])
    msg = update.message or update.callback_query.message
    await msg.reply_text("\n".join(lineas), reply_markup=teclado)

async def cmd_por_tienda(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await _reply(update, "Agrupando por tienda...")
    prendas = fetch_inventario_completo()
    if not prendas:
        await _reply(update, "Inventario vacío o error.")
        return
    tiendas = {}
    for p in prendas:
        t = p["tienda"] or "Sin tienda"
        if t not in tiendas:
            tiendas[t] = {"prendas": [], "inversion": 0, "ganancia": 0}
        tiendas[t]["prendas"].append(p["nombre"])
        tiendas[t]["inversion"] += p["costo"]
        tiendas[t]["ganancia"]  += p["ganancia_real"]
    lineas = ["Resumen por tienda\n"]
    for t, datos in sorted(tiendas.items()):
        lineas.append(f"▫ {t}")
        lineas.append(f"   Prendas:   {len(datos['prendas'])}")
        lineas.append(f"   Inversión: S/{datos['inversion']:.0f}")
        lineas.append(f"   Ganancia:  S/{datos['ganancia']:.0f}\n")
    teclado = InlineKeyboardMarkup([
        [InlineKeyboardButton("⬅ Volver", callback_data="menu_ganancias")],
        [InlineKeyboardButton("✖ Cancelar", callback_data="menu_inicio")],
    ])
    msg = update.message or update.callback_query.message
    await msg.reply_text("\n".join(lineas), reply_markup=teclado)

async def cmd_grafico_stock(update: Update, context: ContextTypes.DEFAULT_TYPE):
    msg = update.message or update.callback_query.message
    await msg.reply_text("Generando gráfico de stock… ⏳")
```

```
    buf = generar_grafico_stock()
    if not buf:
        await msg.reply_text("No hay prendas en el inventario aún.")
        return
    await msg.reply_photo(photo=buf, caption="□ Stock actual por prenda\nVerde=Dispo


# ============================================================
# AUDITORÍA DE GANANCIAS
# ============================================================
def auditar_y_corregir_ganancias() -> dict:
    """Revisa todos los registros de ventas y corrige Ganancia si está mal calculada
    if not NOTION_VENTAS_ID:
        return {"error": "No hay BD de ventas configurada."}
    url     = f"https://api.notion.com/v1/databases/{NOTION_VENTAS_ID}/query"
    payload = {"page_size": 100}
    todos   = []
    # Paginar por si hay más de 100 registros
    while True:
        r = requests.post(url, headers=NOTION_HEADERS, json=payload)
        if r.status_code != 200:
            return {"error": f"Error Notion {r.status_code}"}
        data = r.json()
        todos += data.get("results", [])
        if not data.get("has_more"):
            break
        payload["start_cursor"] = data["next_cursor"]

    revisados  = 0
    corregidos = 0
    errores    = 0
    detalles   = []

    for page in todos:
        try:
            props    = page.get("properties", {})
            page_id  = page["id"]
            cantidad  = props.get("Cantidad",       {}).get("number") or 0
            precio    = props.get("Precio real",    {}).get("number") or 0
            costo_u   = props.get("Costo unitario", {}).get("number") or 0
            descuento = props.get("Descuento",      {}).get("number") or 0
            ganancia_actual = props.get("Ganancia", {}).get("number")
            prenda_r = props.get("Prenda", {}).get("rich_text", [])
            prenda   = prenda_r[0]["text"]["content"] if prenda_r else "?"

            if cantidad == 0 or precio == 0:
                continue  # sin datos suficientes, saltar

            ganancia_correcta = round((precio - descuento - costo_u) * cantidad, 2)
            revisados += 1

            # Solo corrige si la ganancia guardada es mayor a la calculada (bug, no d
            if ganancia_actual is None or (ganancia_actual - ganancia_correcta) > 0.01
                # Corregir en Notion
                patch = requests.patch(
                    f"https://api.notion.com/v1/pages/{page_id}",
```

```
                    headers=NOTION_HEADERS,
                    json={"properties": {"Ganancia": {"number": ganancia_correcta}}}
                )
                if patch.status_code == 200:
                    corregidos += 1
                    detalles.append({
                        "prenda": prenda,
                        "antes": ganancia_actual,
                        "despues": ganancia_correcta,
                        "cantidad": cantidad,
                        "precio": precio,
                        "costo_u": costo_u,
                    })
                else:
                    errores += 1
            except Exception:
                errores += 1
                continue

    return {
        "revisados": revisados,
        "corregidos": corregidos,
        "errores": errores,
        "detalles": detalles,
    }

async def cmd_auditar_ventas(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Comando /auditarventas — revisa y corrige ganancias mal calculadas."""
    msg = update.message or update.callback_query.message
    await msg.reply_text("⏳ Auditando registros de ventas... puede tomar unos segun
    import asyncio, functools
    loop = asyncio.get_event_loop()
    resultado = await loop.run_in_executor(None, auditar_y_corregir_ganancias)

    if "error" in resultado:
        await msg.reply_text(f"❌ {resultado['error']}")
        return

    rev  = resultado["revisados"]
    cor  = resultado["corregidos"]
    err  = resultado["errores"]
    dets = resultado["detalles"]

    if cor == 0:
        await msg.reply_text(
            f"✅ Auditoría completa\n\n"
            f"Registros revisados: {rev}\n"
            f"Correcciones: ninguna necesaria\n"
            f"Todo está bien calculado. 🎯"
        )
        return

    lineas = [
        f"✅ Auditoría completa\n",
        f"Registros revisados: {rev}",
        f"Corregidos:         {cor}",
```

```
        f"Errores:            {err}\n",
        "─── Detalle de correcciones ───",
    ]
    for d in dets:
        antes   = f"S/{d['antes']:.0f}" if d["antes"] is not None else "vacío"
        lineas.append(
            f"• {d['prenda']}\n"
            f"  {d['cantidad']} uds × (S/{d['precio']:.0f} - S/{d['costo_u']:.0f}) "
            f"= S/{d['despues']:.0f}  (antes: {antes})"
        )
    if err > 0:
        lineas.append(f"\n⚠ {err} registro(s) no se pudieron corregir.")

    await msg.reply_text("\n".join(lineas))

# ═══════════════════════════════════════════════════════════
# MAIN — REGISTRO DE HANDLERS
# ═══════════════════════════════════════════════════════════
def main():
    app = ApplicationBuilder().token(TELEGRAM_TOKEN).build()

    # ConversationHandler — IA
    ia_handler = ConversationHandler(
        entry_points=[
            CommandHandler("ia", cmd_ia),
            CallbackQueryHandler(cmd_ia, pattern="^menu_ia$"),
        ],
        states={
            IA_ESPERANDO: [
                MessageHandler((filters.TEXT | filters.PHOTO) & ~filters.COMMAND, ia
            ],
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # ConversationHandler — Nueva prenda guiado
    nueva_prenda_handler = ConversationHandler(
        entry_points=[
            CommandHandler("nueva", cmd_nueva_prenda),
            CallbackQueryHandler(cmd_nueva_prenda, pattern="^menu_nueva_guiado$"),
        ],
        states={
            NP_NOMBRE:      [MessageHandler(filters.TEXT & ~filters.COMMAND, np_rec
            NP_COSTO:       [MessageHandler(filters.TEXT & ~filters.COMMAND, np_rec
            NP_PRECIO:      [
                CallbackQueryHandler(np_recibir_precio_auto, pattern="^np_precio_aut
                MessageHandler(filters.TEXT & ~filters.COMMAND, np_recibir_precio_ma
            ],
            NP_STOCK_TIPO:   [
                CallbackQueryHandler(np_elegir_stock_tipo, pattern="^np_stock_"),
                CallbackQueryHandler(np_volver_precio, pattern="^np_volver_precio$")
            ],
```

```
            NP_STOCK_DOCENAS:[
                CallbackQueryHandler(np_stock_docenas_btn, pattern="^np_doc_"),
                CallbackQueryHandler(np_volver_stock, pattern="^np_volver_stock$"),
            ],
            NP_STOCK_UNICA:  [MessageHandler(filters.TEXT & ~filters.COMMAND, np_sto
            NP_TIENDA:       [
                CallbackQueryHandler(np_recibir_tienda_btn, pattern="^np_tienda"),
                MessageHandler(filters.TEXT & ~filters.COMMAND, np_recibir_tienda_tx
            ],
            NP_FECHA_MES:    [
                CallbackQueryHandler(np_elegir_mes, pattern="^np_mes_"),
                CallbackQueryHandler(np_volver_stock, pattern="^np_volver_stock$"),
            ],
            NP_FECHA_DIA:    [
                CallbackQueryHandler(np_recibir_fecha, pattern="^np_dia"),
                CallbackQueryHandler(np_volver_mes, pattern="^np_volver_mes$"),
                MessageHandler(filters.TEXT & ~filters.COMMAND, np_recibir_fecha),
            ],
            NP_FOTO:         [
                CallbackQueryHandler(np_elegir_foto, pattern="^np_foto_"),
                CallbackQueryHandler(np_volver_dia, pattern="^np_volver_dia$"),
                MessageHandler(filters.PHOTO, np_recibir_foto),
            ],
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # ConversationHandler — Sin foto
    sinfoto_handler = ConversationHandler(
        entry_points=[
            CommandHandler("sinfoto", cmd_sin_foto),
            CallbackQueryHandler(cmd_sin_foto, pattern="^menu_sinfoto$"),
        ],
        states={
            SINFOTO_NOMBRE: [MessageHandler(filters.TEXT & ~filters.COMMAND, sinfoto
            SINFOTO_DATOS:  [MessageHandler(filters.TEXT & ~filters.COMMAND, sinfoto
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # ConversationHandler — Adjuntar foto
    adjfoto_handler = ConversationHandler(
        entry_points=[
            CommandHandler("adjfoto", cmd_adj_foto),
            CallbackQueryHandler(cmd_adj_foto, pattern="^menu_adjfoto$"),
        ],
        states={
            ADJFOTO_BUSCAR:    [MessageHandler(filters.TEXT & ~filters.COMMAND, adjf
```

```
            ADJFOTO_CONFIRMAR: [CallbackQueryHandler(adjfoto_confirmar_prenda)],
            ADJFOTO_RECIBIR:   [MessageHandler(filters.PHOTO, adjfoto_recibir_foto)]
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # ConversationHandler — Venta
    venta_handler = ConversationHandler(
        entry_points=[
            CommandHandler("venta", cmd_vendi),
            CallbackQueryHandler(cmd_vendi, pattern="^menu_venta$"),
        ],
        states={
            VENTA_BUSCAR:    [MessageHandler(filters.TEXT & ~filters.COMMAND, venta_b
            VENTA_CONFIRMAR: [CallbackQueryHandler(venta_confirmar_prenda, pattern="
            VENTA_CANTIDAD:  [MessageHandler(filters.TEXT & ~filters.COMMAND, venta_r
            VENTA_PRECIO:    [
                CallbackQueryHandler(venta_recibir_precio, pattern="^precio_venta_")
                CallbackQueryHandler(venta_volver_cantidad, pattern="^volver_cantidad
                MessageHandler(filters.TEXT & ~filters.COMMAND, venta_recibir_precio
            ],
            VENTA_FECHA:     [
                CallbackQueryHandler(venta_recibir_fecha, pattern="^fecha_venta_"),
                CallbackQueryHandler(venta_volver_descuento, pattern="^volver_descuer
                MessageHandler(filters.TEXT & ~filters.COMMAND, venta_recibir_fecha_r
            ],
            VENTA_CLIENTE:   [
                CallbackQueryHandler(venta_recibir_cliente, pattern="^cliente_sin_nor
                CallbackQueryHandler(venta_recibir_cliente, pattern="^cliente_nueva$"
                CallbackQueryHandler(venta_recibir_cliente, pattern="^cliente_prev:"
                CallbackQueryHandler(venta_volver_fecha, pattern="^volver_fecha$"),
                MessageHandler(filters.TEXT & ~filters.COMMAND, venta_recibir_client
            ],
            VENTA_DESCUENTO: [
                CallbackQueryHandler(venta_recibir_descuento, pattern="^descuento_")
                CallbackQueryHandler(venta_volver_precio, pattern="^volver_precio$")
                MessageHandler(filters.TEXT & ~filters.COMMAND, venta_recibir_descue
            ],
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # ConversationHandler — Consultar prenda
    stock_handler = ConversationHandler(
        entry_points=[
            CommandHandler("prenda", cmd_stock),
            CallbackQueryHandler(cmd_stock, pattern="^menu_stock$"),
        ],
```

```
        states={
            STOCK_BUSCAR:    [MessageHandler(filters.TEXT & ~filters.COMMAND, stock_
            STOCK_CONFIRMAR: [CallbackQueryHandler(stock_confirmar_prenda)],
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # ConversationHandler — Editar
    editar_handler = ConversationHandler(
        entry_points=[
            CommandHandler("actualizar", cmd_editar),
            CallbackQueryHandler(cmd_editar, pattern="^menu_editar$"),
        ],
        states={
            EDITAR_BUSCAR:    [MessageHandler(filters.TEXT & ~filters.COMMAND, edita
            EDITAR_CONFIRMAR: [CallbackQueryHandler(editar_confirmar)],
            EDITAR_CAMPO:     [CallbackQueryHandler(editar_campo)],
            EDITAR_VALOR:     [MessageHandler(filters.TEXT & ~filters.COMMAND, editar
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # ConversationHandler — Eliminar
    eliminar_handler = ConversationHandler(
        entry_points=[
            CommandHandler("eliminar", cmd_eliminar),
            CallbackQueryHandler(cmd_eliminar, pattern="^menu_eliminar$"),
        ],
        states={
            ELIMINAR_BUSCAR:    [MessageHandler(filters.TEXT & ~filters.COMMAND, eli
            ELIMINAR_CONFIRMAR: [CallbackQueryHandler(eliminar_confirmar)],
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # ConversationHandler — Ver foto
    verfoto_handler = ConversationHandler(
        entry_points=[
            CommandHandler("verfoto", cmd_ver_foto),
            CallbackQueryHandler(cmd_ver_foto, pattern="^menu_verfoto$"),
        ],
        states={
            FOTO_BUSCAR:    [MessageHandler(filters.TEXT & ~filters.COMMAND, verfoto
            FOTO_CONFIRMAR: [CallbackQueryHandler(verfoto_confirmar)],
```

```
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # ConversationHandler — Comparar
    comparar_handler = ConversationHandler(
        entry_points=[
            CommandHandler("comparar", cmd_comparar),
            CallbackQueryHandler(cmd_comparar, pattern="^menu_comparar$"),
        ],
        states={
            COMPARAR_BUSCAR: [MessageHandler(filters.TEXT & ~filters.COMMAND, compar
        },
        fallbacks=[
            CommandHandler("cancelar", cmd_cancelar),
            CallbackQueryHandler(fallback_menu_inicio, pattern="^menu_inicio$"),
        ],
        per_message=False,
    )

    # Registrar todos los handlers
    app.add_handler(ia_handler)
    app.add_handler(nueva_prenda_handler)
    app.add_handler(sinfoto_handler)
    app.add_handler(adjfoto_handler)
    app.add_handler(venta_handler)
    app.add_handler(stock_handler)
    app.add_handler(editar_handler)
    app.add_handler(eliminar_handler)
    app.add_handler(verfoto_handler)
    app.add_handler(comparar_handler)

    # Comandos simples
    app.add_handler(CommandHandler("start",       cmd_start))
    app.add_handler(CommandHandler("menu",        cmd_menu))
    app.add_handler(CommandHandler("ayuda",       cmd_ayuda))
    app.add_handler(CommandHandler("cancelar",    cmd_cancelar))
    app.add_handler(CommandHandler("agotados",    cmd_agotados))
    app.add_handler(CommandHandler("limitadas",   cmd_limitadas))
    app.add_handler(CommandHandler("ganancia",    cmd_ganancia))
    app.add_handler(CommandHandler("inventario",  cmd_inventario))
    app.add_handler(CommandHandler("resumen",     cmd_resumen))
    app.add_handler(CommandHandler("pormargen",   cmd_por_margen))
    app.add_handler(CommandHandler("portienda",   cmd_por_tienda))
    app.add_handler(CommandHandler("graficostock",cmd_grafico_stock))
    app.add_handler(CommandHandler("chatid",      cmd_chatid))
    app.add_handler(CommandHandler("auditarventas", cmd_auditar_ventas))

    # Handler genérico para fotos con caption
    app.add_handler(MessageHandler(filters.PHOTO & filters.CAPTION, recibir_foto_nu

    # CallbackQuery para el menú principal
```

```
    app.add_handler(CallbackQueryHandler(manejar_menu, pattern="^menu_"))
    app.add_handler(CallbackQueryHandler(manejar_menu, pattern="^fin_"))

    logger.info("Bot Maricuchis Store v3.0 iniciado con IA Gemini integrada")
    app.run_polling()

if __name__ == "__main__":
    main()
```